A PROJECT REPORT
on

QLens: From a Messy Spreadsheet to a Verified Third-Normal-Form Database

Submitted to Manipal University Jaipur
Towards the partial fulfillment for the Award of the Degree of

B. Tech Computer Science and Engineering

2026–2027

By

Aditya Gaur
Reg. No. 2502051500

Aditya Choudhary
Reg. No. 2502051502

Semester III, Section K3 · Relational Database Management Systems

[University logo: figures/muj-logo.png, 4.56 in × 0.9 in]

Under the guidance of

Ms. Surabhi Agrawal

Department of Artificial Intelligence and Machine Learning
Manipal University Jaipur

Jaipur, Rajasthan

Abstract

Small offices keep their records in a single, ever-growing spreadsheet. A coaching institute's fee register repeats every teacher's phone number on hundreds of rows, spells the same student three different ways, and needs an hour of filtering to answer "who has not paid for October?". These are the update, insertion and deletion anomalies that normalization theory was invented to remove, but the people who suffer from them have never heard the terms and the tools they use do not help: database front-ends expect the user to design the tables, and chat-style assistants answer one question without leaving a correct database behind.

QLens is a local web application that takes such a sheet and produces a normalized PostgreSQL database from the data itself. The file is copied unchanged into a raw table and a typed staging table, with column types inferred by SQL. Candidate keys and functional dependencies are then discovered inside the database by PL/pgSQL functions (`qlens.check_fd`, `qlens.is_unique`) that group the staged rows with dynamic SQL; a dependency that holds on all but a few rows is reported as a data inconsistency together with the rows that break it. Near-duplicate spellings are found with `pg_trgm` trigram similarity over a GIN index, and every repair is logged. From the accepted exact dependencies QLens computes a minimal cover and runs Bernstein's 3NF synthesis, extended with a key relation so that the decomposition is always lossless, and turns the result into tables with surrogate keys, `UNIQUE NULLS NOT DISTINCT` natural keys and foreign keys. The design is applied in a single transaction and accepted only if a view that joins the new tables back together returns exactly the staged rows, compared as multisets with `EXCEPT ALL` in both directions.

QLens keeps its own metadata in a normalized `qlens` schema of 22 tables with CHECK constraints, generated columns, triggers and views that use window functions. Each workspace has its own schemas and its own low-privilege PostgreSQL login role, so user queries are isolated by privileges rather than by convention. On the bundled 3,000-row fee register the analysis finds the two composite candidate keys, six core dependencies and the 13 planted faulty rows, and proposes the textbook four-table design (`students`, `batches`, `teachers`, `fee_register`). The algorithms are verified by unit tests on textbook examples, by 900 randomized synthesis cases checked against a chase-based lossless-join oracle and a brute-force 3NF test, and by end-to-end tests that execute the generated SQL on randomly generated sheets. The query sandbox, performance laboratory, step-through visualizer and assistant are specified in this report and marked with their build status.

Keywords: functional dependency discovery, third normal form, Bernstein synthesis, lossless-join decomposition, PostgreSQL, PL/pgSQL, role-based isolation, data cleaning, `pg_trgm`.

Table of Contents

Abstract i

Table of Contents ii

List of Figures iv

List of Tables vi

List of Listings and Algorithms vii

1 Introduction 1

1.1 Motivation and Problem Statement 1

1.1.1 The fee register 1

1.1.2 The anomalies in the language of normalization 1

1.1.3 Why existing tools do not close the gap 2

1.2 Objectives 3

1.3 Scope of the Work 3

1.3.1 In scope 3

1.3.2 Out of scope 4

1.3.3 Status of the deliverable 4

1.4 Product Scenarios 5

1.4.1 Scenario 1: the coaching-institute owner 5

1.4.2 Scenario 2: the kirana shop 6

1.4.3 Scenario 3: the developer migrating legacy CSV 6

1.4.4 Scenario 4: the student learning normalization 6

1.5 Organisation of the Report 7

2 Requirement Analysis 8

2.1 Functional Requirements 8

2.2 Non-functional Requirements 10

2.3 Use Case Scenarios 10

2.3.1 Detailed use case: normalise the fee register 13

2.4 Other Software Engineering Methodologies 14

2.4.1 Iterative, test-driven development 14

2.4.2 Migration-based schema management 14

2.4.3 Test plan and results 15

3 System Design 17

3.1 Design Goals 17

3.2 System Architecture . . . 18
3.3 Detailed Design Methodologies . . . 20
3.3.1 Metadata Schema . . . 20
3.3.2 Import and SQL Type Inference . . . 24
3.3.3 Dependency and Key Discovery in SQL . . . 26
3.3.4 Data Clean-up . . . 29
3.3.5 Third Normal Form Synthesis . . . 31
3.3.6 Applying a Design and Verifying It is Lossless . . . 35
3.3.7 Security and Isolation . . . 36
3.3.8 Triggers and Views . . . 37

4 Conclusion . . . 40
4.1 What Was Achieved . . . 40
4.2 What Was Learnt . . . 40
4.3 Limitations . . . 41
4.4 Future Work . . . 42

References . . . 44

A Appendix . . . 45
A.1 Complete DDL of Selected Tables . . . 45
A.2 Sample Datasets . . . 47
A.2.1 Coaching fee register . . . 47
A.2.2 Kirana store sales . . . 48
A.2.3 Hostel mess attendance . . . 48
A.3 Screenshots . . . 48
A.4 Test Inventory . . . 49

List of Figures

Figure 1.1 QLens workspace dashboard (dark theme) 5

Figure 2.1 Importing a sheet: inferred types and preview 10
Figure 2.2 Use-case diagram. The three target groups specialise a signed-in user and inherit the common flow (import, analyse, clean, design, apply, query); each group adds the use cases it motivates. Colours follow the pipeline phases of Figure 3.4. 12

Figure 3.1 Layered architecture of QLens. Application code reaches PostgreSQL through two kinds of connection: the shared pool as the owning role, and a per-query sandbox connection as the workspace's own low-privilege LOGIN role. 18
Figure 3.2 Entity–relationship diagram of the `qlens` metadata schema, part 1: accounts, workspaces, the imported dataset and the analysis results (candidate keys, functional dependencies, near-duplicates and the audit log of fixes). Derived from `001_schema.sql`. 21
Figure 3.3 Entity–relationship diagram of the `qlens` metadata schema, part 2: proposed normalized designs, saved queries and run history, English questions, stored execution plans and index experiments. Dashed relationships are optional (nullable foreign keys). 22
Figure 3.4 The normalization pipeline from upload to the lossless check. Left: the steps in order (colour marks the phase); right: the tables each step writes. Steps 4–6 can loop: every clean-up re-profiles the edited columns and re-runs discovery. 27
Figure 3.5 Analyze: dependencies, keys and data issues 30
Figure 3.6 Analyze after accepting all suggestions 31
Figure 3.7 Worked example on the bundled fee register. (a) The columns of the sheet with the dependencies QLens finds; arrows point from determinant to dependent, ↔ marks a pair of mutually determining columns. (b) The schema produced by the synthesis after the 13 faulty rows are cleaned: each dependency moves to the table of its determinant, so a teacher's phone number is stored once instead of on every row. Row counts follow from the sample's construction (300 students, 8 batches, 6 teachers). . . 34

Figure 3.8 Sequence of a user query through the sandbox. The query never runs on the application's pooled connection: it runs on a separate login as the workspace role, whose privileges reach only that workspace's schemas, inside a read-only transaction with a statement timeout and a row cap. The run is then recorded on the pooled connection, where a trigger maintains the saved query's statistics. (Query runner specified; the role, its settings, `query_runs` and the trigger are built and tested.) 36

Figure A.1 Sign-in screen 48
Figure A.2 Dashboard, light theme 49
Figure A.3 Analyze, light theme 50

List of Tables

Table 2.1 Functional requirements and their implementation status. 9
Table 2.2 Non-functional requirements, the mechanism that meets each, and the measured or checked result. 11
Table 2.3 Principal use cases. 13
Table 2.4 Backend test plan. 15

Table 3.1 Technology stack 20
Table 3.2 Data dictionary of the key metadata tables (abridged to the most significant columns) 23
Table 3.3 Selected indexes and the queries they serve 25
Table 3.4 Status of the designed components 39

Table A.1 Backend test files and the number of test functions in each. 50

List of Listings and Algorithms

3.1 The two partial unique indexes on proposals (from `001_schema.sql`) 24
3.2 Safe date parsing used by type inference (from `003_ingest.sql`) 26
3.3 The dependency-measuring function (from `004_analysis.sql`, comments shortened) 28
3.4 Design of the lossless check (illustrative; the join is generated from the proposal) 35
3.5 The usage-counter trigger (from `002_logic.sql`) 37
3.6 Timing trend of saved queries with window functions (from `002_logic.sql`) . 38
A.1 Workspaces, datasets and dataset columns. 45
A.2 Functional dependencies and their left-hand-side columns. 46
A.3 Schema proposals with the lossless-join result. 46
A.4 Query history. 46

3.1 Lossless 3NF synthesis used by QLens (Bernstein synthesis with equivalence merging, the step-5 redundancy pruning and a key relation) 32

CHAPTER 1

Introduction

This report describes QLens, a locally hosted web application that takes an untidy spreadsheet and turns it into a verified relational database in third normal form (3NF). The application reads the data itself, works out which columns determine which others, proposes a decomposition, and creates the tables, keys and foreign keys in PostgreSQL. It also provides a place to query the result and to study how those queries execute. This chapter explains the problem that motivated the work, states the objectives, fixes the boundaries of what was attempted, and walks through four concrete scenarios in which the system is used.

1.1 Motivation and Problem Statement

1.1.1 The fee register

Consider a coaching institute with roughly three thousand fee payments recorded over a year. The office keeps them in a single spreadsheet. Each row holds the student's name and phone number, the batch the student attends, the teacher of that batch together with the teacher's phone number, the month, the amount paid and the mode of payment. The sheet is convenient because every fact about a payment is visible on one line, and nobody had to design anything to start using it.

The trouble appears after a few months. The same sheet now stores each teacher's phone number once for every payment of every student in that teacher's batches. When a teacher changes number, the clerk must find and edit hundreds of rows, and any row that is missed leaves the sheet contradicting itself. A student whose name was typed as *Riya Sharma* in one month and *riya sharma* in another is counted as two people, so the totals for that student are split. Finally, a question as ordinary as "who has not paid for October?" cannot be answered by any single filter, because the answer is the set of students who are *absent* from the October rows. In practice the clerk works it out by hand, which takes an hour and is easy to get wrong.

None of these problems is a software defect in the spreadsheet program. They are structural consequences of storing several different kinds of facts in one table, and relational database theory has had precise names for them for fifty years.

1.1.2 The anomalies in the language of normalization

Codd introduced the relational model and the idea of removing redundancy by decomposing relations into normal forms [1]. The standard treatments [2, 3] classify the damage done by redundancy into three anomalies, all of which the fee register exhibits.

- **Update anomaly.** A single fact is stored in many rows, so a change must be applied to

all of them. Teacher phone numbers repeat in every payment row of every student of that teacher.

- **Insertion anomaly.** A fact cannot be recorded until an unrelated fact exists. A new teacher with no enrolled student, or a new batch with no payment yet, has nowhere to live in the sheet.
- **Deletion anomaly.** Removing one fact destroys another. Deleting the only payment row of the last student in a batch also deletes the record that the batch exists and who teaches it.

These anomalies are explained by *functional dependencies* (FDs). In the register, a student determines a phone number, a student determines a batch, a batch determines a teacher, and a teacher determines a phone number. Writing this with the notation used throughout the report, the four dependencies are $\{\texttt{student_name}\} \rightarrow \texttt{student_phone}$, $\{\texttt{student_name}\} \rightarrow \texttt{batch}$, $\{\texttt{batch}\} \rightarrow \texttt{teacher}$ and $\{\texttt{teacher}\} \rightarrow \texttt{teacher_phone}$. The key of the sheet as a whole is the pair (student, month), since a student pays once per month. Both `teacher` and `teacher_phone` therefore depend on only *part* of that key, and `teacher_phone` depends on it only *transitively*, through the batch and the teacher. A relation in which a non-key attribute depends on part of a key violates second normal form, and one with a transitive dependency violates third normal form. Decomposing the relation so that every determinant is a key of its own table removes the anomalies.

Textbook exercises hand the student the FDs and ask for the decomposition. A real spreadsheet comes with no such list. The dependencies have to be discovered from the data, and the data is dirty. In the sample register that accompanies this project, eight rows carry a spelling variant of a student's name, and one teacher's old phone number survives in five rows after the number was changed. A strict test would reject $\{\texttt{student_name}\} \rightarrow \texttt{student_phone}$ because of those rows, even though the intended dependency clearly holds. A usable tool must therefore distinguish a dependency that is absent from one that is *almost* present, and report the offending rows to the user as inconsistencies to be repaired.

1.1.3 Why existing tools do not close the gap

Spreadsheet-like database products such as Airtable, NocoDB and Baserow give a database look and feel but leave the schema design entirely to the user, who is exactly the person that does not know what a functional dependency is. General-purpose chat tools can answer questions about an uploaded file, but they leave no lasting, constrained, correct database behind, and their answers are not backed by keys or foreign keys. Academic normalization calculators accept FDs typed by hand and never touch data. No widely available tool takes a raw sheet and returns a normalized schema with the data already moved and checked. QLens is an attempt to supply that missing step, with the database engine itself doing as much of the analysis as possible.

1.2 Objectives

The overall aim is to build a working system that converts a flat spreadsheet into a verified 3NF PostgreSQL database and makes the result queryable and explainable. The aim is broken into the following objectives.

1. Import a CSV or XLSX file into a staging table and infer an SQL type, a null count and a distinct count for every column.
2. Discover functional dependencies and candidate keys from the data *inside the database*, using grouped aggregation in SQL, and tolerate a small proportion of violating rows so that dependencies broken only by data-entry errors are still found.
3. Detect near-duplicate text values using trigram similarity in PostgreSQL, and let the user apply fixes with a recorded audit trail.
4. Compute a 3NF decomposition with a lossless-join guarantee, by synthesis from a minimal cover [4], and present it with plain-language reasons and generated DDL.
5. Apply the accepted design in a single transaction, creating tables, primary keys, foreign keys and uniqueness constraints, migrating the data, and verifying that joining the new tables reproduces the original rows.
6. Provide a sandboxed, read-only query workspace with saved queries and a complete run history.
7. Provide a performance laboratory that exposes the real `EXPLAIN ANALYZE` plan and index suggestions, and a visualizer that steps through a `SELECT` stage by stage.
8. Keep user data separate: every account owns workspaces, and every workspace is confined to its own schemas under its own database role.

A second, equally important objective concerns the report itself. The design deliberately relies on database features rather than application code wherever possible: `CHECK`, `UNIQUE` and foreign key constraints, generated columns, partial unique indexes, triggers, views with window functions, PL/pgSQL functions, GIN indexes, and role-based privileges. The metadata schema of the application is itself a normalized design, and the later design chapters analyse it.

1.3 Scope of the Work

The system is intended for a single machine and a single analyst working with a table of up to roughly one hundred thousand rows. The following lists record precisely what is included and what is not.

1.3.1 In scope

- Import of a single-sheet CSV or XLSX file, including day-first dates, through an SQL type-inference step.

- Discovery of exact and approximate functional dependencies, and of single and composite candidate keys, performed by SQL and PL/pgSQL functions on the staged data.
- Near-duplicate detection with `pg_trgm` and a GIN index, and value-level inconsistency fixes recorded in an audit log.
- Functional-dependency algebra (attribute closure, minimal cover) and Bernstein 3NF synthesis, with generated names and DDL.
- A relational metadata schema of 21 tables with constraints, generated columns, triggers and reporting views.
- Local accounts with salted password hashing and server-side sessions, and a PostgreSQL role and pair of schemas for each workspace.
- Bundled sample datasets with planted defects: a coaching-institute fee register of 3,000 rows, a kirana shop sales register and a hostel mess register.
- A web interface in a consistent dark and light design system.
- Application of a design in one transaction followed by a lossless-join check, and an interactive entity–relationship canvas.
- A read-only SQL editor, saved queries and run history; a performance laboratory; a step-through visualizer; and a natural-language assistant.

1.3.2 Out of scope

- Database engines other than PostgreSQL.
- Multi-sheet workbooks, and files beyond about 20 MB or 100,000 rows.
- Write statements from the query editor. The editor is read-only; only the application's own flows (apply a design, merge duplicates, create an index, scale up data) change user tables.
- Step-through of statements that are not a plain `SELECT`, or of queries dominated by common table expressions or window functions.
- Live multi-user collaboration, mobile applications and payments.
- Hardening for public hosting on the open internet. The sandbox is designed to be sound for a local demonstration, and exposing it publicly would be a separate piece of work.
- Normal forms beyond 3NF, such as BCNF and 4NF, and the discovery of multivalued dependencies.

1.3.3 Status of the deliverable

The report describes the system as it stood at the time of writing, and the status of each capability is marked in the same way throughout. Built means implemented and covered by automated tests (212 backend tests and the frontend test suite pass), In progress means partly implemented, and Planned means designed but not yet built. The core analytic pipeline from

import through 3NF proposal is built and tested. Applying a design transactionally with the lossless check, the design canvas and the query sandbox interface are in progress, and the performance laboratory, the visualizer and the language-model assistant are planned. No feature is described in this report as working unless it is marked Built.

1.4 Product Scenarios

Four short scenarios show how the system is expected to be used, who uses it, and how much of each scenario QLens supports today. A summary of the product as the user first meets it is shown in Figure 1.1.

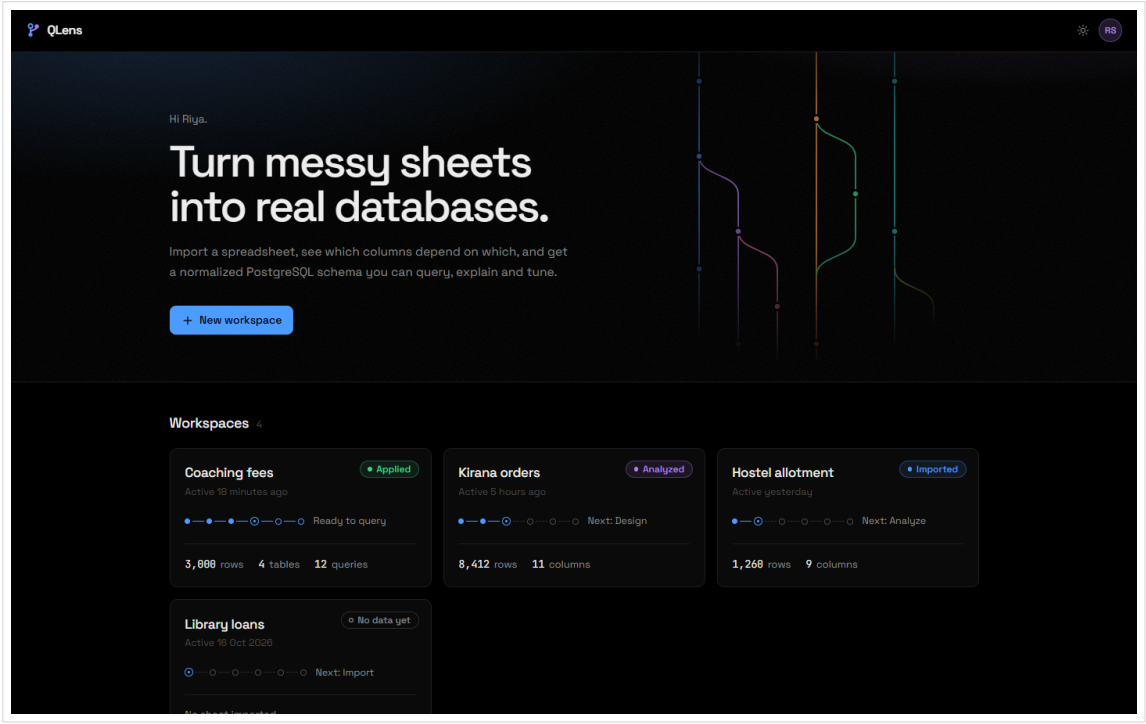


Figure 1.1: QLens workspace dashboard (dark theme)

1.4.1 Scenario 1: the coaching-institute owner

Mrs. Verma runs a coaching institute and records all fees in one sheet. She creates an account, opens the bundled fee register (or uploads her own file), and presses *Analyze*. QLens infers column types and finds the dependencies of Section 1.1: it reports, for example, that a student determines a batch, a batch determines a teacher, and a teacher determines a phone number. Because eight rows spell a student's name differently and one teacher's earlier number remains in five rows, a few of those dependencies hold for all rows but a handful; QLens lists the violating rows as inconsistencies and proposes fixes. After she accepts them, the system proposes four tables: `students`, `batches`, `teachers` and `fee_register`, with foreign keys between them and a uniqueness constraint on the student and month of each payment. A teacher's phone number is now stored once. Creating these tables in her workspace and verifying the

migration is the apply step, which is currently In progress.

Step	What QLens does	Status
Import and type inference	Stages the file, infers SQL types	Built
Dependency and key discovery	Runs in SQL, about 0.3 s for 3,000 rows	Built
Duplicate detection and fixes	`pg_trgm` similarity, audit log	Built
3NF proposal	Bernstein synthesis, names and DDL	Built
Apply and lossless check	One transaction, `EXCEPT ALL` comparison	In progress

1.4.2 Scenario 2: the kirana shop

A neighbourhood shop keeps a daily sales sheet with the date, item, item category, supplier, supplier phone, quantity and price. The owner wants to know which supplier's items sell best, but the same supplier's phone number is repeated on hundreds of lines and a few lines carry a typing variant of an item name. Using the bundled kirana dataset, QLens discovers that an item determines its category and supplier, and that a supplier determines the phone, and proposes the corresponding split. Once the data is in separate tables, a question such as total quantity sold per supplier becomes a short `JOIN` with `GROUP BY`. The analysis part of this scenario is Built; the read-only query editor in which the owner would run that question is In progress, and asking it in plain English through the assistant is Planned.

1.4.3 Scenario 3: the developer migrating legacy CSV

A developer inherits a decade-old CSV export from a system with no schema documentation and must load it into an application database. Rather than guess keys by eye, the developer uploads the file and reads off the candidate keys and dependencies that actually hold in the data, including composite keys of up to three columns. The proposed design arrives with generated `CREATE TABLE` statements carrying primary keys, foreign keys and `UNIQUE` constraints, which can be copied into the application's own migrations. The percentages of violating rows reported next to each approximate dependency tell the developer where the legacy data cannot be trusted. Discovery and DDL generation are Built; the in-database migration and its `EXCEPT ALL` check are In progress, so a developer today should treat the generated DDL as a reviewed starting point and not as an automatic migration.

1.4.4 Scenario 4: the student learning normalization

A second-year student is asked to normalize a relation to 3NF and cannot tell whether the answer is right. Using one of the sample files, the student sees the dependencies, the candidate keys and the minimal cover derived from them, and can compare the proposal with a hand-worked solution. Because the dependencies come from a data set that the student already understands, each table in the proposal can be traced back to a determinant, which makes the reasoning about partial and transitive dependencies concrete. Later, the planned

visualizer will show how a `SELECT` over the resulting tables passes through `FROM`, `WHERE`, `GROUP BY`, `HAVING` and `ORDER BY`, with row counts at each stage, and the planned performance laboratory will show the corresponding execution plan. The analysis and the 3NF proposal are Built; the visualizer and the laboratory are Planned.

1.5 Organisation of the Report

The remaining chapters follow the order in which the system was specified and built. The requirements chapter lists the users, the functional and non-functional requirements and the use cases. The chapter on software engineering methodology describes how the work was planned and tested. The system design chapters then present the goals, the architecture, the relational schema with its entity–relationship diagrams, and the detailed design of the analysis algorithms, the sandbox, and the user interface, with screenshots of the features that exist. The report closes with a conclusion that restates the results honestly against the objectives above, followed by references and an appendix containing selected SQL listings and test information.

CHAPTER 2

Requirement Analysis

This chapter fixes what QLens must do and how well it must do it. The requirements come from the problem described in the introduction: a small business keeps its records in one wide spreadsheet, and the same facts are typed many times with small variations. The target users are not database administrators. They are a coaching-centre owner, a shop keeper or a hostel manager who can read a table but has never written a `CREATE TABLE` statement. Each requirement below is therefore written from the point of view of what such a person must be able to do, and each carries a status that reflects the state of the implementation at the time of writing: Built means built and covered by automated tests, In progress means partly built, and Planned means designed but not yet implemented.

2.1 Functional Requirements

The functional requirements follow the pipeline of Figure 3.4: a sheet enters the system, is analysed in SQL, is proposed as a normalised design, is applied, and is then queried. Requirements FR-1 to FR-8 form the normalisation pipeline; FR-9 to FR-13 are the surrounding services that make the resulting database usable.

Import and type inference (FR-1, FR-2). Figure 2.1 shows the import screen. Raw cells are first stored as text in the source schema `ws_<id>_src`, so nothing is lost if a guess is wrong. Type inference then runs in SQL: for each column the system counts how many values succeed under each candidate cast, using helper functions such as `qlens.try_date`, and picks the narrowest type for which every non-empty value succeeds. Because Indian spreadsheets commonly write dates day-first, `try_date` tries `DD/MM/YYYY` before any month-first reading.

Discovery and cleanup (FR-3 to FR-5). Dependency checking is expressed as a grouped aggregate: a dependency $\{X\} \rightarrow A$ holds if no group of rows sharing the same X contains more than one distinct value of A. The PL/pgSQL functions `qlens.check_fd` and `qlens.is_unique` implement this test, and a search over small determinant sets uses them to find candidate keys and minimal dependencies. For the 3,000-row fee register the whole analysis takes about 0.3 s. The cleanup step exists because real data violates the dependencies it ought to satisfy; the planted errors in the sample register (eight rows with spelling variants of a student name, and five rows carrying a superseded phone number for Mr. Gupta) are exactly the cases that stop a clean dependency from being discovered.

Table 2.1: Functional requirements and their implementation status.

ID	Requirement	Status
FR-1	**Import.** Accept CSV and XLSX files, detect the header row and delimiter, and stage the raw cells in a per-workspace source schema without losing any row.	Built
FR-2	**Type inference.** Infer a SQL type for every column (`integer`, `numeric`, `date`, `boolean`, `text`) by testing the data in SQL, including day-first dates such as `05/03/2026`, and let the user override the choice.	Built
FR-3	**FD and key discovery.** Discover the functional dependencies that hold in the data and the candidate keys, using queries executed inside PostgreSQL rather than in application memory.	Built
FR-4	**Near-duplicate detection.** Find values that differ only by spelling, case or spacing (for example `Rahul Sharma` and `rahul sharma`) using trigram similarity, and present them as groups with a suggested canonical value.	Built
FR-5	**Inconsistency fixes.** Detect cases where a determinant maps to more than one dependent value (an old and a new phone number for the same student), let the user choose the surviving value, and record every change in an audit log.	Built
FR-6	**3NF proposal.** Compute a minimal cover of the discovered dependencies, synthesise a third-normal-form decomposition, choose primary keys, substitute surrogate keys for composite references, name the tables and generate the DDL.	Built
FR-7	**Apply with lossless check.** Create the proposed tables, load the data and verify, inside one transaction, that the natural join of the new tables reproduces the cleaned source exactly; roll everything back if it does not.	In progress
FR-8	**Design canvas.** Show the proposed schema as an entity-relationship diagram that the user can inspect and edit before applying.	In progress
FR-9	**Sandboxed querying.** Let the user run arbitrary SELECT statements against the new tables, with write access impossible and runaway queries stopped by the database.	In progress
FR-10	**Saved queries and history.** Keep a named, versioned list of saved queries and a log of every executed statement with its duration and row count. The metadata schema and views exist; the interface is being built.	In progress
FR-11	**Performance lab.** Show `EXPLAIN ANALYZE` output as a plan tree, highlight the slowest nodes and suggest indexes.	Planned
FR-12	**Step-through visualiser.** Animate how a query is evaluated, clause by clause, on a small sample of the data, for teaching purposes.	Planned
FR-13	**Agentic chat.** Offer a Gemini-powered assistant that can inspect the workspace and propose, but not silently execute, schema or query changes.	Planned
FR-14	**Accounts and workspaces.** Register and log in users; give every user isolated workspaces, each with its own schemas and database role.	Built
FR-15	**Sample data.** Provide generated example datasets (a coaching fee register, kirana shop sales, a hostel mess register) so that the tool can be demonstrated without private data.	Built

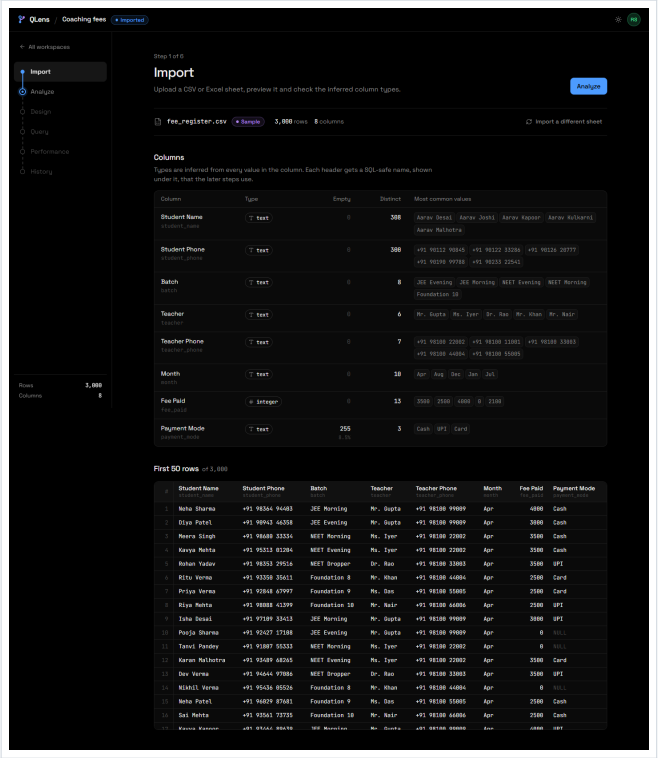


Figure 2.1: Importing a sheet: inferred types and preview

Proposal and apply (FR-6, FR-7). The proposal uses the synthesis algorithm of Bernstein [4], described in the design chapter, which guarantees a dependency-preserving, lossless-join decomposition in third normal form. The apply step re-checks that guarantee against the real rows rather than trusting the algorithm: it compares the cleaned source with the join of the new tables using `EXCEPT ALL` in both directions, so that duplicate rows are counted and not collapsed.

2.2 Non-functional Requirements

Security deserves a short justification because the product, by design, lets a user type SQL. A single check in the application (for example rejecting statements that begin with anything other than `SELECT`) is easy to bypass with a comment, a semicolon or a function call that writes. QLens therefore enforces the restriction at several layers that fail independently: the database role cannot write outside its own schemas, the transaction is declared `READ ONLY`, a time limit stops an accidental cross join, and the result is truncated at 1,000 rows before it is serialised. The design follows the usual advice for multi-tenant PostgreSQL and the privilege model described in the PostgreSQL documentation [5].

2.3 Use Case Scenarios

Figure 2.2 shows the actors and the interactions that QLens supports. There are two human actors. The *owner* is the non-technical user who imports a sheet and wants a clean database.

Table 2.2: Non-functional requirements, the mechanism that meets each, and the measured or checked result.

ID	Quality	Requirement and mechanism	Result
NFR-1	Security and isolation	A user's SQL must never reach another user's data or change the database. Each workspace has its own PostgreSQL `LOGIN` role with privileges only on its schemas; query sessions run `READ ONLY`, with `statement_timeout` of 5 s and a 1,000-row cap on returned results. Passwords are hashed with argon2; session tokens are stored only as SHA-256 hashes and sent in an `httpOnly` cookie.	Role isolation and auth covered by `test_workspaces.py` and `test_auth.py`.
NFR-2	Performance	Analysis must feel interactive on sheets of a few thousand rows.	Dependency and key discovery on 3,000 rows ≈ 0.3 s; near-duplicate search over 5,000 distinct values ≈ 1 s using a GIN trigram index.
NFR-3	Correctness and losslessness	The produced schema must be in 3NF, preserve the dependencies and reproduce the original data exactly.	Minimal cover, closure and Bernstein synthesis property-tested on thousands of random dependency sets; runtime `EXCEPT ALL` check at apply time.
NFR-4	Integrity	The database, not only the application, must reject invalid metadata.	21 metadata tables with `CHECK`, `UNIQUE`, foreign keys, generated columns and partial unique indexes; checked by `test_schema.py`.
NFR-5	Auditability	Every data change made by the cleanup step must be traceable and reversible in principle.	Each fix is stored in `dataset_fixes` with the old value, new value and user.
NFR-6	Usability	Users with no SQL background must understand each screen. Plain-language labels, a pure-black dark theme and a light theme, and one consistent typeface (Space Grotesk).	Both themes implemented and captured in the screenshots in this report.
NFR-7	Portability and locality	The tool must run on one student laptop with no cloud service.	A private PostgreSQL 18 cluster is started and migrated by the application; no data leaves the machine except in the optional assistant.
NFR-8	Maintainability	Schema and code must evolve without manual steps and	Numbered SQL migrations (001–004) applied by the



Figure 2.2: Use-case diagram. The three target groups specialise a signed-in user and inherit the common flow (import, analyse, clean, design, apply, query); each group adds the use cases it motivates. Colours follow the pipeline phases of Figure 3.4.

The *analyst* (a student or an employee who knows some SQL) queries the result. PostgreSQL and the optional Gemini service act as supporting systems.

Table 2.3: Principal use cases.

ID	Actor	Goal	Main flow	Tables touched
UC-1	Owner	Create an account and a workspace	Register, log in, create a workspace; a role and two schemas are provisioned	`users`, `sessions`, `workspaces`
UC-2	Owner	Import a spreadsheet	Upload file, review inferred types and preview, confirm	`datasets`, `dataset_columns`, `ws_<id>_src`
UC-3	Owner	Find and fix data errors	Run analysis, review near-duplicate groups and conflicting values, accept fixes	`dataset_fixes`, `functional_dependencies`
UC-4	Owner	Normalise the fee register	Accept the 3NF proposal and apply it (detailed below)	`schema_proposals`, `ws_<id>` tables
UC-5	Analyst	Query the clean data	Write SELECT, run in the sandbox, see rows and duration	`query_runs`
UC-6	Analyst	Save and revisit a query	Name the query, save, reopen and compare its run times	`saved_queries`, `query_runs`
UC-7	Analyst	Understand a slow query	Open the performance lab, read the plan tree, apply a suggested index	`explain_snapshots`, `index_experiments`
UC-8	Owner	Ask for help in plain language	Type a request in the chat; the assistant proposes an action for approval	`nl_questions`

2.3.1 Detailed use case: normalise the fee register

Actor: owner of a coaching centre. **Precondition:** logged in, workspace created, the 3,000-row fee register imported (UC-2). **Postcondition:** the workspace holds four related 3NF tables whose join equals the cleaned register.

1. The owner opens the Analyze screen and starts the analysis. The system computes dependencies and keys in SQL (about 0.3 s) and lists the issues found.
2. The system reports the near-duplicate groups. Eight rows contain spelling variants of one student name; the owner confirms the suggested canonical spelling.
3. The system reports that one student, Mr. Gupta, has two different phone numbers; five rows hold the old number. The owner selects the current number and the system rewrites those rows, writing one entry per change to `dataset_fixes`.

4. The system re-runs discovery on the cleaned data. Dependencies that were previously blocked now hold: {student_name} → student_phone, batch, {batch} → teacher and {teacher} → teacher_phone.
5. The system computes the minimal cover, synthesises the relations and displays the proposal:
   - students(id, student_name, student_phone, batch_id);
   - batches(id, batch, teacher_id);
   - teachers(id, teacher, teacher_phone);
   - fee_register(id, student_id, month, fee_paid, payment_mode) with UNIQUE(student_id, month).
6. The owner reviews the generated DDL and the diagram and chooses *Apply*.
7. In a single transaction the system creates the four tables, inserts the rows, and runs the EXCEPT ALL comparison between the join of the new tables and the cleaned source. If both differences are empty the transaction commits.
8. If the comparison finds any difference, the transaction is rolled back, the workspace is left unchanged, and the owner is told which rows differ.

Alternative flows: at step 2 or 3 the owner may reject a suggestion, in which case the dependency stays unsatisfied and the proposal keeps the affected attributes together in a wider relation; at step 6 the owner may rename tables or columns, which only changes the generated DDL.

2.4 Other Software Engineering Methodologies

2.4.1 Iterative, test-driven development

The project was built in short iterations, each delivering one vertical slice that could be run and shown: first the database and migrations, then authentication and workspaces, then import, then analysis, then the proposal. Within an iteration the work was divided into small tasks. For each task a failing test was written first, the minimum code was added to make it pass, and the result was reviewed against the written specification before the next task started. Review caught real defects, for example in the handling of connection commits and in how generated names collided, and each such defect was turned into a regression test (test_conn_commit.py, test_naming.py). The sequence of iterations kept the riskiest part, the algebra of dependencies and the claim of losslessness, in the earliest tested code.

2.4.2 Migration-based schema management

The database schema is not edited by hand. It is defined by numbered SQL files that a runner applies in order and records in a version table: 001_schema.sql (tables and constraints), 002_logic.sql (triggers, helper functions, the check_fd and is_unique functions), 003_ingest.sql

(staging and type inference) and `004_analysis.sql` (analysis support and views). A fresh clone therefore reaches exactly the same schema as the developer's machine, and `test_migrate.py` verifies that running the migrations twice is harmless.

2.4.3 Test plan and results

Table 2.4 lists the backend test files, grouped by the component they verify. Most tests run against a real PostgreSQL instance created for the test session, because the central logic of the system lives in SQL and cannot be meaningfully tested against a mock.

Table 2.4: Backend test plan.

Test file (`backend/tests`)	What it verifies
`test_migrate.py`	Migrations apply in order to an empty cluster and are idempotent.
`test_schema.py`	Metadata constraints: `CHECK`, `UNIQUE`, foreign keys, generated columns, partial unique indexes, the `touch_updated_at` and `record_query_run` triggers, and the three reporting views.
`test_auth.py`	Registration, argon2 verification, hashed session tokens, cookie behaviour, rejection of bad credentials.
`test_workspaces.py`	Per-workspace role and schema creation, isolation between workspaces, `READ ONLY` and timeout enforcement.
`test_conn_commit.py`	Connection helper commits and rolls back correctly.
`test_parse.py`	CSV and XLSX parsing, header and delimiter detection, odd encodings and empty cells.
`test_stage.py`	Staging of raw cells and SQL type inference, including day-first dates.
`test_ingest_routes.py`	Upload endpoints and error responses.
`test_samples.py`	Sample generators are deterministic and contain the planted errors (8 name variants, 5 stale phone rows).
`test_discover.py`	Dependency and candidate-key discovery on known tables, including the fee register.
`test_cleanup.py`	Near-duplicate grouping, inconsistency detection, fixes, audit log and accept-all.
`test_fd_algebra.py`	Attribute closure, minimal cover and Bernstein synthesis, including property tests over thousands of random dependency sets that assert every output relation is in 3NF and that the decomposition is lossless and dependency-preserving.
`test_naming.py`	Table and column naming rules and collision handling.
`test_proposal.py`	3NF proposal end to end: foreign-key substitution, key choice, generated DDL executes and rebuilds the original rows.

Results. At the time of writing, 212 backend tests pass in the latest full run, including the property-based tests of third normal form and losslessness. The React interface is tested with `vitest`, which exercises the formatting helpers, the API client and the main components; this suite also passes. Tests for the apply step, the query sandbox interface and the later features

(performance lab, visualiser, assistant) are written together with those features and are not yet part of the totals.

CHAPTER 3

System Design

This chapter describes how QLens is built and, more importantly, why it is built that way. The database is not an afterthought in this project: the metadata schema, the dependency discovery, the clean-up queries and the lossless-join check are all expressed in PostgreSQL, and the application code around them is deliberately thin. Where a component is still being finished at the time of writing, its status is marked with Built, In progress or Planned in the same way as in the previous chapter.

3.1 Design Goals

Five goals shaped every decision below. They are listed in the order in which they win when two of them conflict.

Correctness and losslessness. A normalized database is only useful if it holds exactly the information the spreadsheet held. The system therefore never rewrites the user's original rows: the imported data is kept untouched in a `_raw` table, all clean-up happens in a `_staging` copy with an audit trail, and a design is marked *applied* only after the tables rebuilt from it reproduce the staging table exactly. This is enforced by a database `CHECK` constraint and not merely by application code (Section 3.3.6).

Visible SQL. The target user is a learner or a small-business owner who will eventually have to read SQL. Every dependency, every merge and every generated table is therefore backed by a statement the user can inspect: the discovery functions are ordinary PL/pgSQL, the generated `CREATE TABLE` script is shown before it runs, and the query screens display the SQL next to the result.

Isolation. Several workspaces share one PostgreSQL cluster, and users type arbitrary SQL. A workspace must not be able to read another workspace's data, change its own data from the query box, or monopolise the server. This is solved with database privileges, per-workspace roles and timeouts rather than by filtering SQL text (Section 3.3.7).

Explainability. Each suggestion carries a sentence in plain language ("Each `batch` has exactly one `teacher`"), the measure behind it (rows that violate it) and a way to reject it. The same principle applies to normalization itself: each proposed table is accompanied by the dependency that justifies it.

Polish. A tool meant for non-specialists must feel finished: a consistent design system with a pure-black dark theme and a light theme, meaningful empty and error states, and responses that are fast enough to feel interactive (the 3,000-row fee register is analysed in about 0.3 seconds).

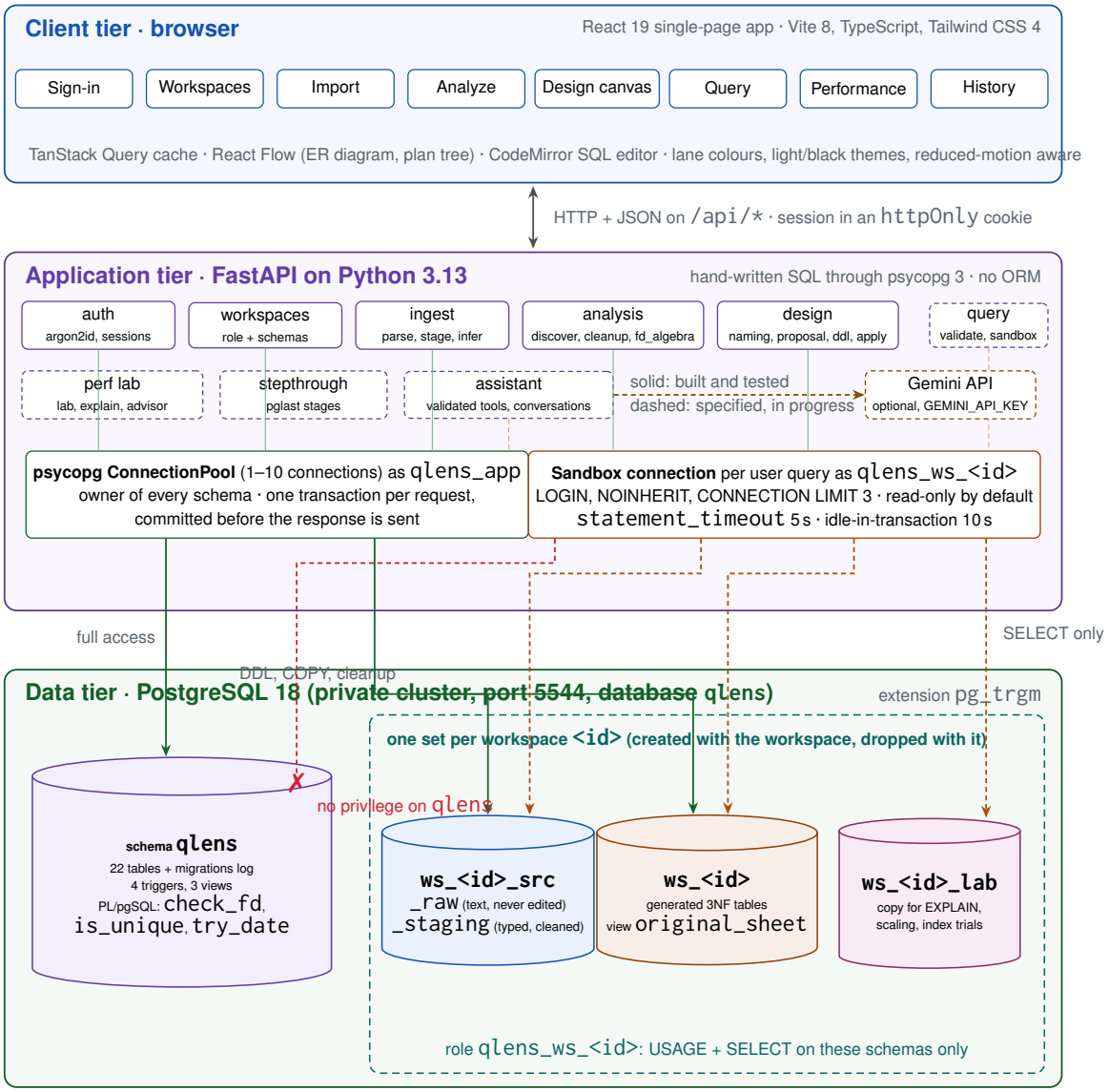


Figure 3.1: Layered architecture of QLens. Application code reaches PostgreSQL through two kinds of connection: the shared pool as the owning role, and a per-query sandbox connection as the workspace's own low-privilege LOGIN role.

3.2 System Architecture

QLens is a conventional three-tier web application whose bottom tier does unusually much work. Figure 3.1 shows the layers; the paragraphs below describe each of them from the browser downwards.

Presentation layer. The user interface is a single-page application written in React and TypeScript and bundled with Vite [6]. It keeps no authoritative state of its own: every screen (Import, Analyze, Design, Query) is a view over data fetched from the API, and a page reload reconstructs it from the server. The session is carried by an `httpOnly`, `SameSite=Lax` cookie, so scripts running in the page cannot read it.

Application layer. The API is a FastAPI service [7] organised as one router per concern (authentication, workspaces, ingest, analysis, design, query). Routers validate input with typed request models, call a service module and return typed response models. The services are plain functions that take a database connection; they contain orchestration but very little computation, because the computation is done in SQL.

Connection handling. Application connections are drawn from a `psycopg` 3 pool [8] (between one and ten connections). A FastAPI dependency opens one transaction per request, commits on success and rolls back on any exception. The dependency is declared with function scope, which makes FastAPI run its exit code, and therefore the `COMMIT`, *before* the response is sent. With the default request scope the commit would run after the client had already received a success status, so a failing commit would be invisible and an immediate follow-up request could miss the write. Committing first removes that class of error.

Sandbox connections. User-written queries do not use the pool above. For each workspace the application opens a connection as that workspace's own PostgreSQL role (created with the workspace, Section 3.3.7), which can read only the workspace's schemas and is read-only by default. The role's password is derived from a server secret with HMAC, so it never has to be stored.

Data layer. All state lives in one PostgreSQL 18 cluster [5] that the project starts privately on a non-default port. It contains four kinds of schema:

- `qlens`: the metadata schema owned by the application (21 tables, 3 views, 2 trigger functions, the discovery functions `check_fd`, `is_unique` and `try_date`);
- `ws_<id>`: the user's normalized tables after a design has been applied, and the only schema that queries run against by default;
- `ws_<id>_src`: the imported data as two tables, `_raw` (as parsed, never modified) and `_staging` (typed, cleaned, with a surrogate `_row_id`);
- `ws_<id>_lab`: a copy of the normalized tables used by the performance lab, so that index experiments never touch the main schema (Planned).

Why logic sits in the database. Discovering functional dependencies means grouping the whole table once per candidate; moving 100,000 rows into Python for each of several hundred candidates would dominate the running time and would also hide the logic from a reader who knows SQL. Keeping the work in PL/pgSQL lets the planner use hash aggregation, keeps the data inside the server, and makes each measurement a query that can be re-run by hand. The only algorithms implemented outside SQL are the ones that operate on *sets of dependencies*

Table 3.1: Technology stack

Layer	Technology	Role
Database	PostgreSQL 18, `pg_trgm`, PL/pgSQL	Storage, dependency discovery, similarity search, constraints
Backend	Python 3.13, FastAPI, Pydantic	HTTP API, validation, orchestration
DB driver	psycopg 3 with `psycopg_pool`	Parameterised queries, pooling, safe identifier quoting
Security	argon2-cffi, `hashlib` SHA-256	Password hashing, session-token hashing
Import	`csv`, openpyxl	CSV and XLSX parsing
Frontend	React, TypeScript, Vite	Single-page interface
Testing	pytest, randomised property tests, Vitest	212 backend tests, frontend unit tests

rather than rows (closure, minimal cover, synthesis), because they are small, pure and easy to property-test.

3.3 Detailed Design Methodologies

3.3.1 Metadata Schema

The `qlens` schema stores everything the application knows *about* the user's data. It is split into five areas: accounts (`users`, `sessions`); workspaces and the imported dataset (`workspaces`, `datasets`, `dataset_columns`); analysis results (candidate keys, dependencies, duplicate groups, the fix log); the proposed design (`schema_proposals`, `proposed_tables`, `proposed_columns`, unique keys); and query history with the performance lab (`saved_queries`, `query_runs`, `nl_questions`, `explain_snapshots`, `index_experiments`). Figures 3.2 and 3.3 show the entity–relationship diagram in two parts.

The schema follows a small set of conventions, stated at the top of the migration file so they are applied uniformly. Surrogate keys are `bigint GENERATED ALWAYS AS IDENTITY`; the exceptions are the session table (its natural key is the token hash) and the junction tables (composite keys). Every timestamp is `timestamptz NOT NULL DEFAULT now()`. Closed vocabularies are `text` with a `CHECK (x IN (...))` constraint, which is easier to evolve than an enumerated type. Every column that later becomes a SQL identifier (column names, proposed table names) is constrained to the pattern `^[a-z_][a-z0-9_]{0,62}$`, so it is always safe to quote and always fits PostgreSQL's 63-byte identifier limit.

Table 3.2 gives a data dictionary for the six tables that carry the most design weight; the remaining tables are variations on the same patterns.

Constraint highlights. A few constraints carry real design intent and are worth spelling out.

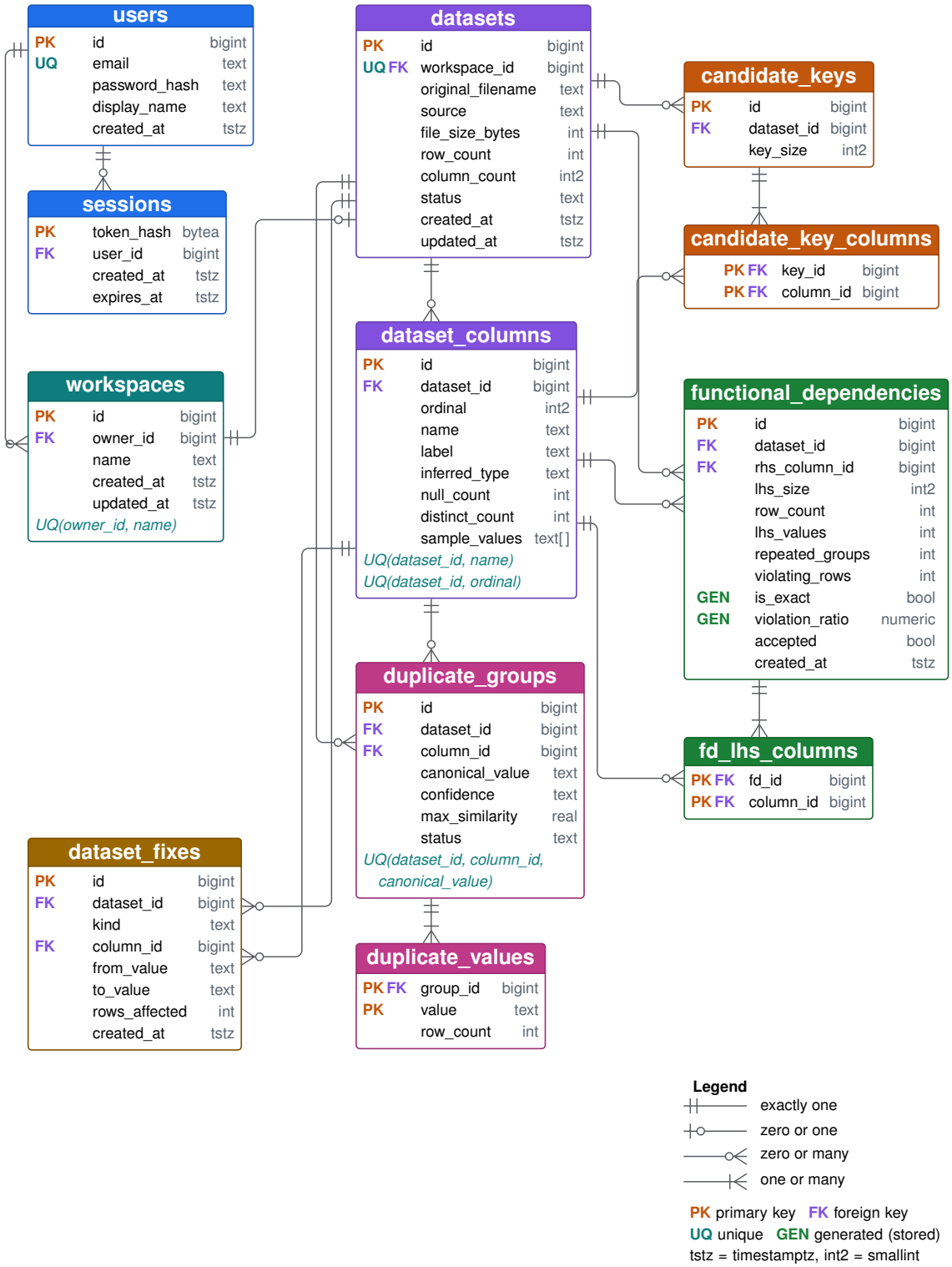


Figure 3.2: Entity–relationship diagram of the `qlens` metadata schema, part 1: accounts, workspaces, the imported dataset and the analysis results (candidate keys, functional dependencies, near-duplicates and the audit log of fixes). Derived from `001_schema.sql`.

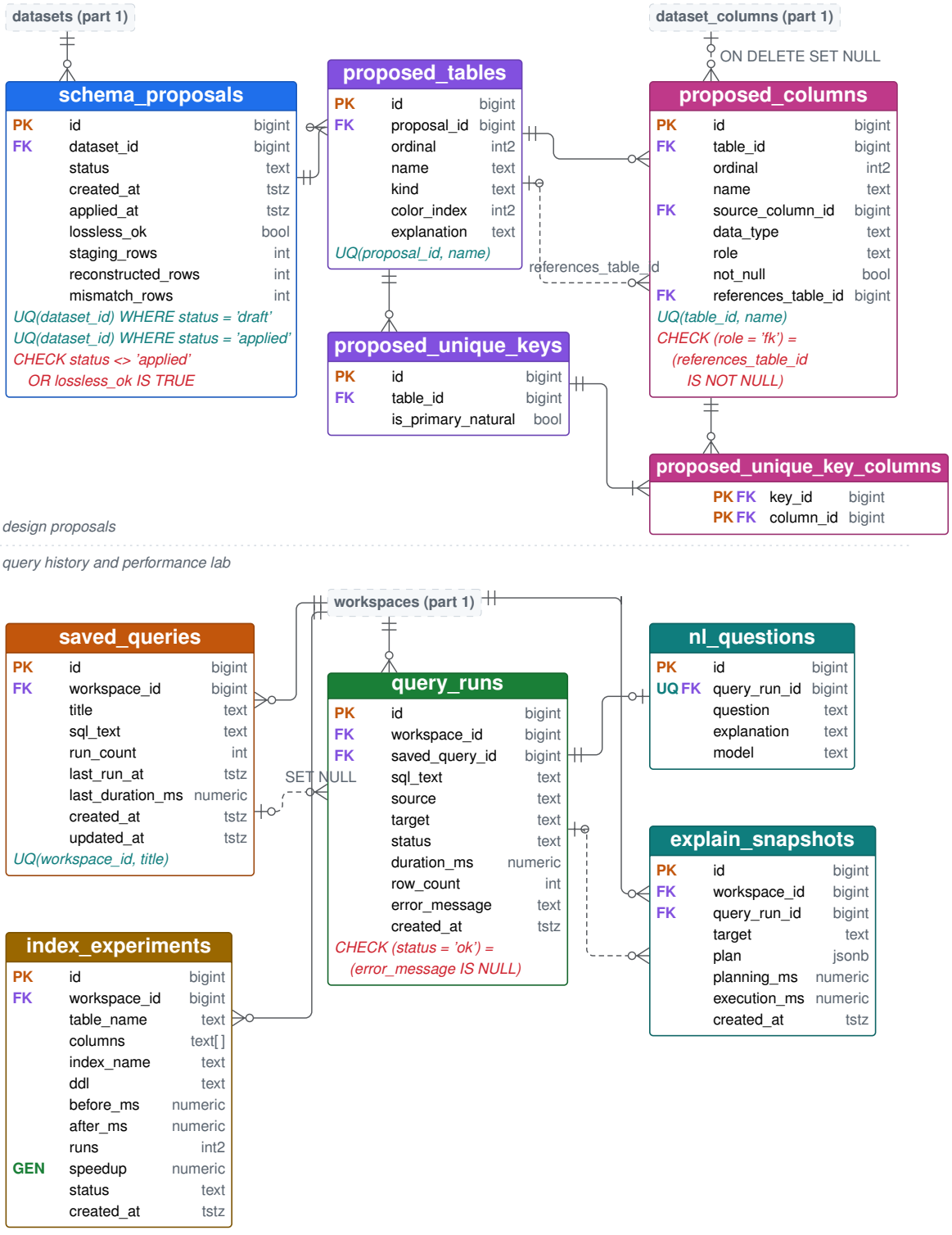


Figure 3.3: Entity–relationship diagram of the `qlens` metadata schema, part 2: proposed normalized designs, saved queries and run history, English questions, stored execution plans and index experiments. Dashed relationships are optional (nullable foreign keys).

Table 3.2: Data dictionary of the key metadata tables (abridged to the most significant columns)

Table	Column	Type	Constraint and meaning
datasets	workspace_id	bigint	UQ, FK workspaces; one dataset per workspace
	status	text	CHECK IN ('imported','analyzed','applied')
	column_count	smallint	CHECK BETWEEN 1 AND 60
dataset_columns	name	text	identifier pattern; UQ with dataset_id
	inferred_type	text	integer, numeric, boolean, date or text
	null_count, distinct_count	integer	import-time profile used to prune the search
functional_dependencies	rhs_column_id	bigint	FK dataset_columns; LHS in fd_lhs_columns
	violating_rows	integer	rows that must change for the FD to hold; ≤ row_count
	is_exact	boolean	GEN violating_rows = 0
	violation_ratio	numeric	GEN violating_rows / NULLIF(row_count,0)
	accepted	boolean	DEFAULT true; false once the user rejects it
duplicate_groups	canonical_value	text	spelling all members merge into
	confidence	text	same_normalized or similar
	status	text	pending, merged or dismissed
schema_proposals	status	text	draft, applied or superseded
	lossless_ok	boolean	result of the EXCEPT ALL comparison
	mismatch_rows	integer	rows differing in either direction
query_runs	source, target	text	typed, english, stepthrough, perf; main or lab
	status	text	ok, error or timeout
	error_message	text	NULL exactly when status = 'ok'

- *Generated columns.* `is_exact` and `violation_ratio` in `functional_dependencies`, and `speedup` in `index_experiments`, are `GENERATED ALWAYS AS (...) STORED`. They are derived from the stored counts, so no code path can write a dependency that claims to be exact while recording violations. The division uses `NULLIF(row_count, 0)` so an empty table gives `NULL` and not an error.
- *Partial unique indexes.* A dataset may have at most one draft proposal and at most one applied proposal, but any number of superseded ones. Two partial unique indexes express this directly (Listing 3.1); the applied design therefore stays applied while a new draft is being edited.
- *The lossless gate.* `schema_proposals` carries `CHECK (status <> 'applied' OR lossless_ok IS TRUE)`. The `IS TRUE` matters: a plain `lossless_ok` would evaluate to `NULL` for a proposal that was never checked, and a `CHECK` constraint accepts `NULL`. With `IS TRUE` an unchecked proposal cannot be marked applied.
- *Biconditional checks.* In `proposed_columns`, `CHECK ((role = 'fk') = (references_table_id IS NOT NULL))` states that a column is a foreign key exactly when it references a table. In `query_runs`, a run has an error message exactly when it did not succeed.
- *Cascades and* `SET NULL`. Deleting a user, workspace or dataset cascades down through every dependent table, so no orphan rows can survive. `SET NULL` is used only where history must outlive its parent: deleting a saved query keeps its runs, and losing a source column does not delete the design column derived from it.

Listing 3.1: The two partial unique indexes on proposals (from `001_schema.sql`)

```
CREATE UNIQUE INDEX schema_proposals_one_draft_idx
    ON qlens.schema_proposals (dataset_id) WHERE status = 'draft';
CREATE UNIQUE INDEX schema_proposals_one_applied_idx
    ON qlens.schema_proposals (dataset_id) WHERE status = 'applied';
```

Index justification. PostgreSQL does not index foreign-key columns automatically, and an unindexed foreign key makes every cascading delete a sequential scan of the child table. Each foreign-key column is therefore either the leading column of a `UNIQUE` or primary-key index, or has an explicit index with a comment stating what it serves. Table 3.3 lists the indexes that exist for a reason beyond the cascade.

3.3.2 Import and SQL Type Inference

The importer reads CSV or XLSX, normalises each header to a safe identifier (lower case, runs of non-alphanumeric characters replaced by underscores) while remembering the original label for display, and loads every cell as text into `_raw`. Loading as text first is deliberate: it can never fail on a malformed value, and it keeps an exact record of what the user uploaded.

Types are then inferred *in SQL*, from the data. For each column the importer asks

Table 3.3: Selected indexes and the queries they serve

Index	Definition	Serve
`workspaces_owner_id_name_key`	`UNIQUE (owner_id, name)`	Works list of one user; re- jects du- pli- cate name
`candidate_keys_dataset_idx`	`(dataset_id, key_size)`	Keys of a datase small- est first
`dataset_fixes_dataset_idx`	`(dataset_id, created_at DESC)`	Edit his- tory, newes first
`query_runs_workspace_time_idx`	`(workspace_id, created_at DESC)`	Histor page, works sum- mary, `v_slo`
`query_runs_saved_query_time_idx`	`(saved_query_id, created_at) WHERE saved_query_id IS NOT NULL`	`v_sav` par- tial be- cause most runs are ad hoc
`proposed_columns_references_idx`	`(references_table_id) WHERE NOT NULL`	Relati lines in the di- a- gram
`sessions_user_id_idx`	`(user_id)`	Log out ev- ery-

PostgreSQL whether every non-empty value matches a pattern for, boolean, integer, numeric and date patterns; the narrowest type that every value satisfies is chosen, and `text` is the fallback. The same query computes the profile that later steps depend on: null count, distinct count and a few sample values.

Dates are the delicate case. A value that merely *looks* like a date may not be one (`31/02/2026`), and casting it would raise an error and abort the whole import. The function `qlens.try_date` (Listing 3.2) therefore returns `NULL` instead of failing. It accepts ISO dates and day-first dates, which is a deliberate choice: the sample data and the target users write `dd/mm/yyyy`, so `03/04/2026` is read as 3 April. The inference query wraps the call in a `CASE` so that the PL/pgSQL function runs only on values that already match a date-shaped regular expression, which avoids paying a function call for every cell of every text column.

Listing 3.2: Safe date parsing used by type inference (from `003_ingest.sql`)

```
CREATE FUNCTION qlens.try_date(p text) RETURNS date
LANGUAGE plpgsql STABLE AS $$
BEGIN
    IF p ~ '^\d{4}-\d{2}-\d{2}$' THEN
        RETURN p::date;
    ELSIF p ~ '^\d{1,2}[/-]\d{1,2}[/-]\d{4}$' THEN
        RETURN to_date(replace(p, '-', '/'), 'DD/MM/YYYY');
    END IF;
    RETURN NULL;
EXCEPTION WHEN others THEN
    RETURN NULL;
END;
$$;
```

Finally the typed table `_staging` is created from `_raw` using the inferred types (casting dates through `try_date`) and given a `_row_id` identity column. All later analysis and clean-up operate on `_staging`.

3.3.3 Dependency and Key Discovery in SQL

The analysis stage turns a flat table into a list of candidate keys and functional dependencies [1]. Figure 3.4 shows where it sits in the overall pipeline. The design rests on one observation: whether a dependency $X \rightarrow A$ holds, and how badly it is broken, can be read off a single grouped query.

The measuring function. Listing 3.3 is `qlens.check_fd`. For a table, a list of left-hand-side columns and one right-hand-side column it returns four numbers. The inner query `c` counts rows per (LHS, RHS) pair. The outer query `g` then groups by LHS alone, keeping the total `n` and the count of the most common RHS value `keep`. If $X \rightarrow A$ holds exactly, every group has a single RHS value and `n = keep`. Summing `n - keep` over groups gives the number of rows that would have to be changed for the dependency to hold, which is a g_3-style error measure: the minimum fraction of rows to delete or correct so that the dependency becomes

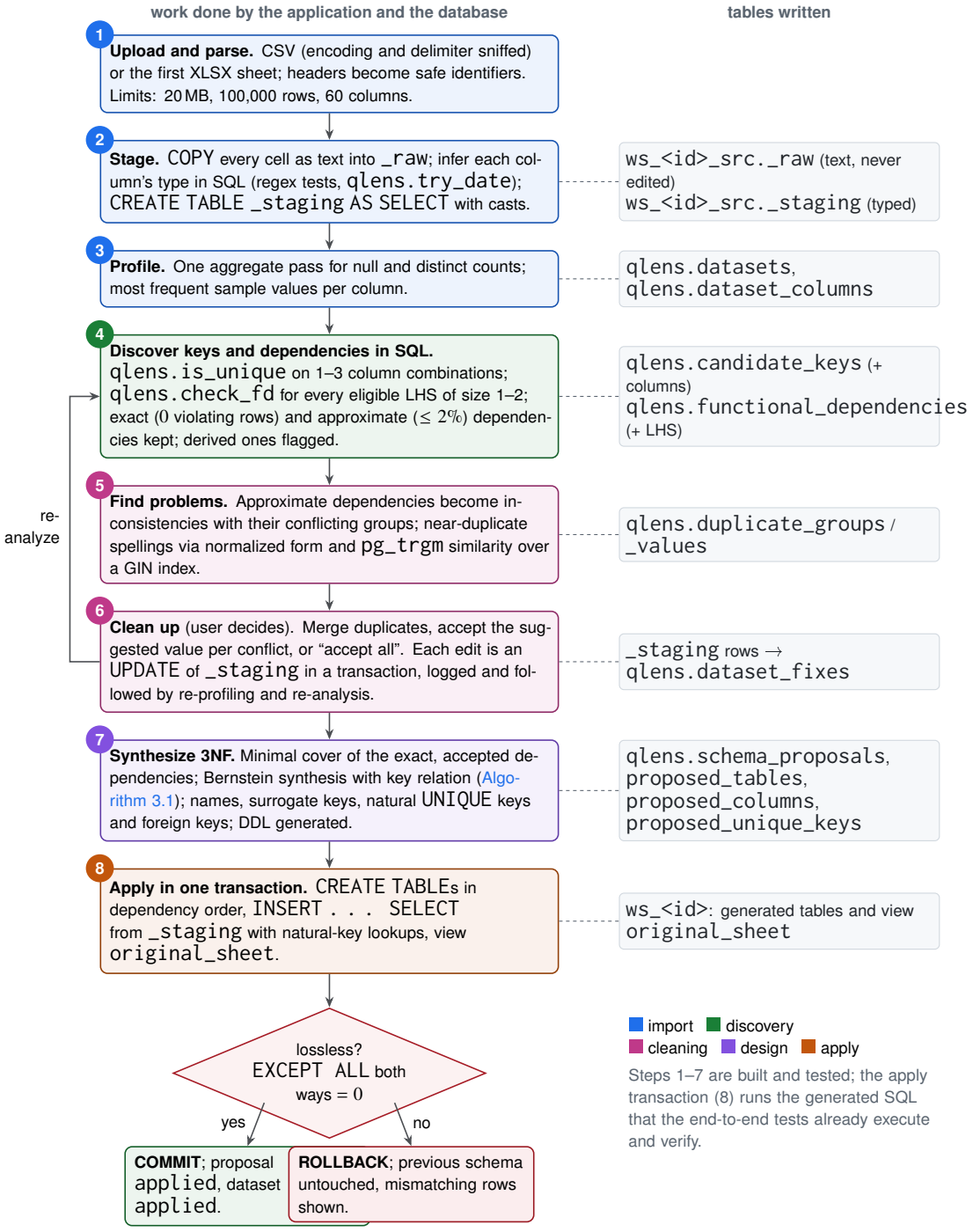


Figure 3.4: The normalization pipeline from upload to the lossless check. Left: the steps in order (colour marks the phase); right: the tables each step writes. Steps 4–6 can loop: every clean-up re-profiles the edited columns and re-runs discovery.

exact. Schema and table names enter the dynamic SQL only through `format('%I')`, so they are always correctly quoted identifiers.

Listing 3.3: The dependency-measuring function (from `004_analysis.sql`, comments shortened)

```
CREATE FUNCTION qlens.check_fd(p_schema text, p_table text, p_lhs text[], p_rhs text)
RETURNS TABLE (row_count bigint, lhs_values bigint, repeated_groups bigint, violating_rows
    bigint)
LANGUAGE plpgsql STABLE AS $$
DECLARE
    t   text := format('%I.%I', p_schema, p_table);
    lhs text;
BEGIN
    SELECT string_agg(format('%I', c), ', ') INTO lhs FROM unnest(p_lhs) AS c;
    RETURN QUERY EXECUTE format(
        'WITH c AS (SELECT %1$s, %2$I, count(*) AS cnt FROM %3$s GROUP BY %1$s, %2$I), '
        '     g AS (SELECT %1$s, sum(cnt) AS n, max(cnt) AS keep FROM c GROUP BY %1$s) '
        'SELECT (SELECT count(*) FROM %3$s), count(*), count(*) FILTER (WHERE n >= 2), '
        '       coalesce(sum(n - keep), 0)::bigint FROM g',
        lhs, p_rhs, t);
END;
$$;
```

The returned `repeated_groups` is the number of LHS values that occur in at least two rows. It exists because a dependency that was never tested against a repeated value proves nothing: if every `student_name` occurred exactly once, `student_name` → anything would hold trivially. `NULL` is treated as an ordinary value (`GROUP BY` puts all nulls in one group), which matches how the normalized design will store it.

Candidate keys. `qlens.is_unique` counts the rows and the distinct combinations of a column list and returns whether they are equal. The key search tries single columns, then pairs, then triples, and before calling the database it discards combinations that cannot possibly be keys using the import-time profile: a combination that contains any nulls is rejected (a key must be filled in on every row); and if the product of the columns' distinct counts is smaller than the row count the combination cannot be unique. Supersets of an already-found key are not reported again. Triples are tried only when no smaller key exists and the table is narrow.

Candidate dependencies. Single-column left-hand sides are tried for every pair of columns; two-column left-hand sides are tried only when at most 20 columns are eligible, because the number of pairs grows quadratically. A dependency is kept when it has at least two repeated groups and its violating rows number at most 2% of the table (the *approximate threshold*). Exact dependencies and near-misses are stored together; the generated column `is_exact` separates them. The near-misses are the interesting ones for data quality: they are dependencies that *ought* to hold but are broken by a few dirty rows, which is exactly the signature of an inconsistency such as one teacher recorded with two phone numbers.

Noise suppression. Naively, a wide table yields hundreds of true but useless dependencies. Six rules, all applied before anything is stored, keep the list short enough for a person to review.

1. *Eligible determinants only.* Boolean and constant columns cannot be determinants. Numeric columns are treated as *measures* (a fee, a quantity) unless their name ends in `id`, `no`, `num`, `number` or `code`, in which case they are identifiers.
2. *Keys are skipped.* A candidate key determines every other column, so dependencies with a key on the left carry no information and are not tested.
3. *Repeated evidence.* At least two LHS values must repeat (above).
4. *Minimal left-hand sides.* A two-column LHS is not tested against a column that one of its members already determines exactly, and an approximate pair dependency is dropped if one member alone approximately determines the same column. A finer grouping can only have fewer violations, so the pair would merely isolate the dirty rows and masquerade as a clean dependency.
5. *Root cause.* If $X \to Z$ holds exactly and $Z \to A$ nearly holds, the broken $X \to A$ is the same inconsistency seen through Z, so only $Z \to A$ is kept. Because the groups of Z are unions of the groups of X, $Z \to A$ never has fewer violations than $X \to A$, so no comparison of counts is needed. Columns that determine each other are ordered by position so exactly one of them keeps the dependency.
6. *Narrow tables only for pairs.* The 20-column limit above.

Re-analysis. The whole analysis runs in one transaction that deletes and rebuilds the candidate keys and dependencies, then rebuilds the pending duplicate groups. A rejection made by the user is remembered by what the dependency *says* (its LHS set and RHS) and not by its row id, so it survives re-analysis. On the 3,000-row fee register the complete analysis, including the SQL passes and the duplicate search, takes about 0.3 s.

3.3.4 Data Clean-up

Real spreadsheets contain spellings that differ by a character (`Sneha Reddy`, `Sneha Reddi`) and cells that disagree with the rest of their row group. Both would otherwise become violations in the dependency list and block a clean design. The clean-up module finds them and offers a fix; every fix is applied to `_staging` only and recorded in `dataset_fixes` (kind, column, old value, new value, rows affected), so the original upload can always be recovered from `_raw` and every change can be explained.

Near-duplicate detection. Two kinds of group are produced.

- *Certain duplicates* (`same_normalized`): spellings that become equal after trimming, lower-casing and collapsing spaces and full stops (`Mrs. Rao` and `Mrs Rao`).

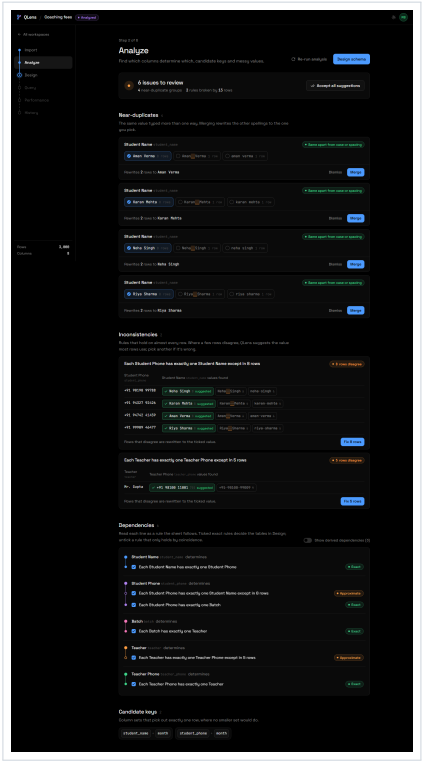


Figure 3.5: Analyze: dependencies, keys and data issues

- *Probable duplicates* (`similar`): spellings whose trigram similarity exceeds 0.5, found with the `pg_trgm` extension [9]. Comparing every pair of d distinct values is quadratic in d. The candidate values are therefore copied into a temporary table with a GIN index on `gin_trgm_ops`, and a self-join using the indexed `%` operator finds the look-alikes of each value without scanning all pairs. For 5,000 distinct values the search finishes in about one second, and 5,000 is the enforced upper limit for a column.

The join is written as a `MATERIALIZED` common table expression so the planner cannot choose a cheap equality test as the join key and pair all digit-free values with each other.

Frequency asymmetry. Similarity alone produces false positives: `Ananya` and `Anaya` may both be real people with ten rows each. Two further rules reduce them. Values whose digits differ are different things (`Foundation 9` against `Foundation 10`, two phone numbers, bills `KB-0001` and `KB-0008`), and a typo is rarer than the correct spelling, so a pair qualifies only if the rarer value has at most one third of the rows of the more frequent one. Qualifying pairs are merged into clusters with a union-find structure, and the canonical spelling of a cluster is its most frequent member. The sample fee register plants eight rows with variant student names (four students, each spelled in lower case or with a doubled space in two rows); the rules propose exactly those four merges and nothing else.

Inconsistencies from approximate dependencies. A near-miss from Section 3.3.3 becomes an *inconsistency*: for each left-hand value whose rows disagree, the screen lists the variants

with their row counts and suggests the majority value. In the sample, Mr. Gupta's old phone number appears in five rows, so `teacher` ⟶ `teacher_phone` is broken in exactly those five rows; the suggested fix rewrites them to the number used in all his other rows. The name variants, in turn, show up as the near-miss `student_phone` ⟶ `student_name` with exactly eight violating rows, until the duplicate merge removes them. The rewrite is a single `WITH old AS (SELECT ... FOR UPDATE), upd AS (UPDATE ... RETURNING ...)` statement that updates the affected rows and returns the old values, which the service logs as it goes. Accepting all suggestions applies every merge and fix in one transaction, after which the columns are re-profiled and the analysis is repeated. Figure 3.6 shows the screen afterwards: the issues list is empty and the dependencies are exact.

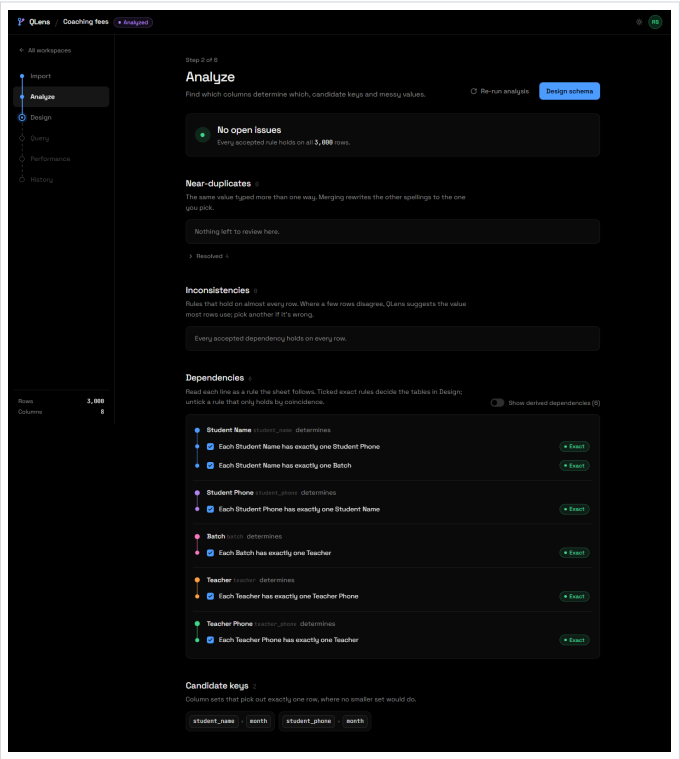


Figure 3.6: Analyze after accepting all suggestions

3.3.5 Third Normal Form Synthesis

Once the accepted exact dependencies are known, the design stage computes a decomposition into third normal form. This part is pure set algebra over attribute names, so it is implemented as a small Python module and tested separately from the database. It follows Bernstein's synthesis algorithm [4], summarised in Algorithm 3.1.

Closure. The closure X^+ of an attribute set under a dependency set F is computed by repeatedly adding the right-hand side of any dependency whose left-hand side is already contained. Closure answers every question the algorithm needs: $X \to A$ is implied by F exactly when $A \in X^+$ [2]; X is a superkey when X^+ is the whole attribute set.

Algorithm 3.1. Lossless 3NF synthesis used by QLens (Bernstein synthesis with equivalence merging, the step-5 redundancy pruning and a key relation)

Input: attributes U in sheet order; exact, accepted dependencies F; candidate keys $K_1, \ldots, K_m$ found in the data

Output: relations $R_1, \ldots, R_n$ (entities) and one fact relation R_f; every R_i is in 3NF and $\bowtie_i R_i$ is lossless

$G \leftarrow \textsc{MinimalCover}(F)$ `// singleton RHS, left-reduced, no redundant FD`
partition G into groups $G_X = \{X \to A \in G\}$, one per left-hand side X
foreach *pair of groups* G_X, G_Y **do**
  if $X \subseteq Y_G^+$ ***and*** $Y \subseteq X_G^+$ **then** `// equivalent determinants`
    merge G_X and G_Y into one group with determinants $\{X, Y\}$
$J \leftarrow \{X \to B \mid X, Y$ determinants of one merged group, $B \in Y\}$
for $X \to A \in G$ ***from last to first*** **do** `// Bernstein step 5`
  if $A \in X^+_{(G \setminus \{X \to A\}) \cup J}$ **then** remove $X \to A$ from G
foreach *merged group with determinants* $X_1, \ldots, X_k$ **do**
  $R \leftarrow X_1 \cup \cdots \cup X_k \cup \{A \mid X_j \to A \in G\}$; key $\leftarrow$ earliest X_j in sheet order
  emit entity relation R unless it carries no dependency
$D \leftarrow$ attributes appearing on the right of some emitted relation's non-key part
if *some emitted* R *contains* $U \setminus D$ ***and*** $R_G^+ = U$ **then**
  mark R as the fact relation R_f
else
  $Z \leftarrow U$
  for $a \in U$ ***in reverse sheet order*** **do** `// FD-key by reverse reduction`
    if $(Z \setminus \{a\})_G^+ = U$ **then** $Z \leftarrow Z \setminus \{a\}$
  $R_f \leftarrow Z$; its unique key is K_1 if $K_1 \subseteq Z$, else none (surrogate key only)
drop every entity relation whose attributes are a subset of another relation's
return *entity relations, then* R_f

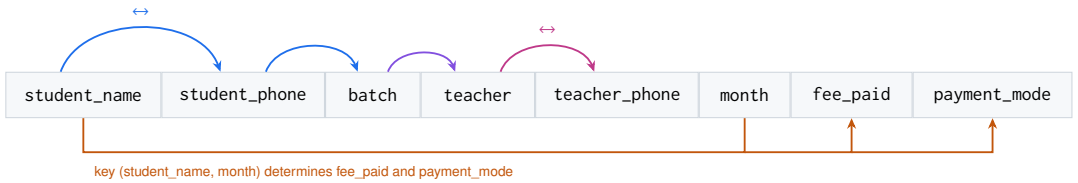
Minimal cover. The set F is reduced to a canonical cover: right-hand sides are split to single attributes; extraneous left-hand attributes are removed (an attribute B in $XB \to A$ is extraneous if $A \in X^+$ already); and redundant dependencies are removed (a dependency is redundant if its right-hand side is in the closure of its left-hand side under the *other* dependencies). All tie-breaks follow the spreadsheet's column order, so the result is deterministic.

Synthesis. Cover dependencies with the same left-hand side are grouped, one relation per group containing the left-hand side and all of its right-hand sides. Groups whose determinants are equivalent ($X \leftrightarrow Y$) are merged, and after merging, dependencies that have become redundant because of the equivalence are pruned, last to first; without this step a merged relation can keep a non-prime attribute that is determined by a non-key and so violates third normal form. Each resulting relation has its determinant as primary key and the equivalent determinants as further unique keys.

Lossless-join guarantee. A dependency-preserving synthesis is not necessarily lossless unless one relation contains a key of the whole table. After grouping, the algorithm therefore checks whether any relation R_i satisfies $R_i^+ \supseteq U$, where U is the full attribute set, and if none does, it adds a relation consisting of a key of U under the accepted dependencies (found by dropping attributes in reverse column order while the remainder still determines everything). In QLens the attributes that no dependency determines form the *fact table*; the fact relation is the synthesized one that contains all of them and whose closure covers U, or else this added key relation. The argument for losslessness is the standard one: starting from the fact relation, each other relation is joined on its determinant, which is a key of that relation, so each join adds attributes functionally determined by attributes already present and can neither add nor remove rows. The property is not only argued but also tested: property-based tests generate thousands of random dependency sets and assert that the output relations are in third normal form, preserve every dependency and contain a key of U.

Worked example. On the cleaned coaching-fee register, discovery reports the exact dependencies {`student_name`} $\to$ `student_phone` (and the reverse, since phone numbers are unique per student), {`student_name`} $\to$ `batch`, {`batch`} $\to$ `teacher`, {`teacher`} $\to$ `teacher_phone`, and the key (`student_name`, `month`) (the near-misses having been removed by the clean-up of Section 3.3.4). The minimal cover removes {`student_name`} $\to$ `teacher` because it follows from the other two. Synthesis yields one relation per determinant, and Figure 3.7 shows the resulting design: `students`(`id`, `student_name` **UQ**, `student_phone` **UQ**, `batch_id` **FK**), `batches`(`id`, `batch` **UQ**, `teacher_id` **FK**), `teachers`(`id`, `teacher` **UQ**, `teacher_phone` **UQ**) and the fact table `fee_register`(`id`, `student_id` **FK**, `month`, `fee_paid`, `payment_mode`, `UNIQUE(student_id, month)`). The proposal step then replaces each determinant by a surrogate key, substitutes foreign keys for the dependent columns, names the tables, and emits the `CREATE TABLE` script for review.

(a) Dependencies discovered in the 3,000-row sheet



Before cleaning: {student_phone} → student_name fails on 8 rows (variant spellings such as riya sharma) and {teacher} → teacher_phone fails on 5 rows (Mr. Gupta's old number). {student_name} → batch and {student_phone} → teacher are derived and not shown.

(b) Synthesized 3NF schema (lane colours as in the application)

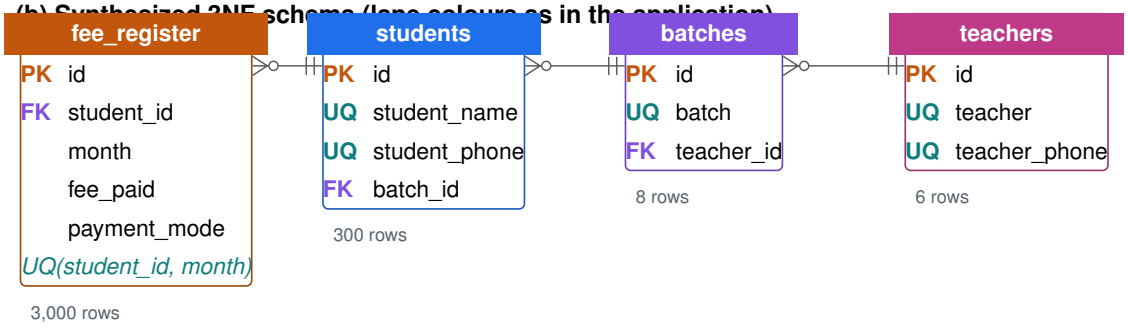


Figure 3.7: Worked example on the bundled fee register. (a) The columns of the sheet with the dependencies QLens finds; arrows point from determinant to dependent, ↔ marks a pair of mutually determining columns. (b) The schema produced by the synthesis after the 13 faulty rows are cleaned: each dependency moves to the table of its determinant, so a teacher's phone number is stored once instead of on every row. Row counts follow from the sample's construction (300 students, 8 batches, 6 teachers).

The phone number of a teacher is stored once instead of in every row of every student the teacher teaches, so the update anomaly described in Chapter 1 disappears.

3.3.6 Applying a Design and Verifying It is Lossless

In progress

Applying a design creates the normalized tables in `ws_<id>` and fills them from `_staging`. This is the step with the highest cost of a mistake, so it is designed around a single rule: the user sees the result of a verification, never an assumption. The design is as follows; the implementation is in progress.

1. The whole operation runs in *one transaction*: create the tables, insert dimension rows with `INSERT ... SELECT DISTINCT`, insert the fact rows by joining staging to the dimensions on their natural keys, and run the check. If anything fails, `ROLLBACK` leaves the workspace as it was and no half-built schema remains.
2. The check rebuilds the original table by joining the new tables back together and compares it with `_staging` *as a multiset*, in both directions, using `EXCEPT ALL` (Listing 3.4). `EXCEPT` without `ALL` would treat the data as a set and hide a lost duplicate row; `EXCEPT ALL` cancels rows one for one, so a count difference of even one row is a mismatch.
3. The three counts (staging rows, reconstructed rows, mismatching rows) are written to `schema_proposals`, and only if `mismatch_rows = 0` is the status set to *applied*. The `CHECK` constraint of Section 3.3.1 makes it impossible to record an applied proposal without a passing check, even through a bug in application code.

Listing 3.4: Design of the lossless check (illustrative; the join is generated from the proposal)

```
WITH rebuilt AS (
    SELECT s.student_name, s.student_phone, b.batch, t.teacher,
           t.teacher_phone, f.month, f.fee_paid, f.payment_mode
    FROM   ws_7.fee_register f
    JOIN   ws_7.students s ON s.id = f.student_id
    JOIN   ws_7.batches  b ON b.id = s.batch_id
    JOIN   ws_7.teachers t ON t.id = b.teacher_id
), only_staging AS (
    SELECT * FROM ws_7_src._staging_cols EXCEPT ALL SELECT * FROM rebuilt
), only_rebuilt AS (
    SELECT * FROM rebuilt EXCEPT ALL SELECT * FROM ws_7_src._staging_cols
)
SELECT (SELECT count(*) FROM only_staging) + (SELECT count(*) FROM only_rebuilt) AS
     mismatch_rows;
```

Here `_staging_cols` stands for the staging table restricted to the data columns (the surrogate `_row_id` is excluded, since it has no counterpart in the design). After a successful apply the main schema is granted to the workspace role and the dataset status becomes `applied`; a later redesign creates a new draft while the applied design remains in force.

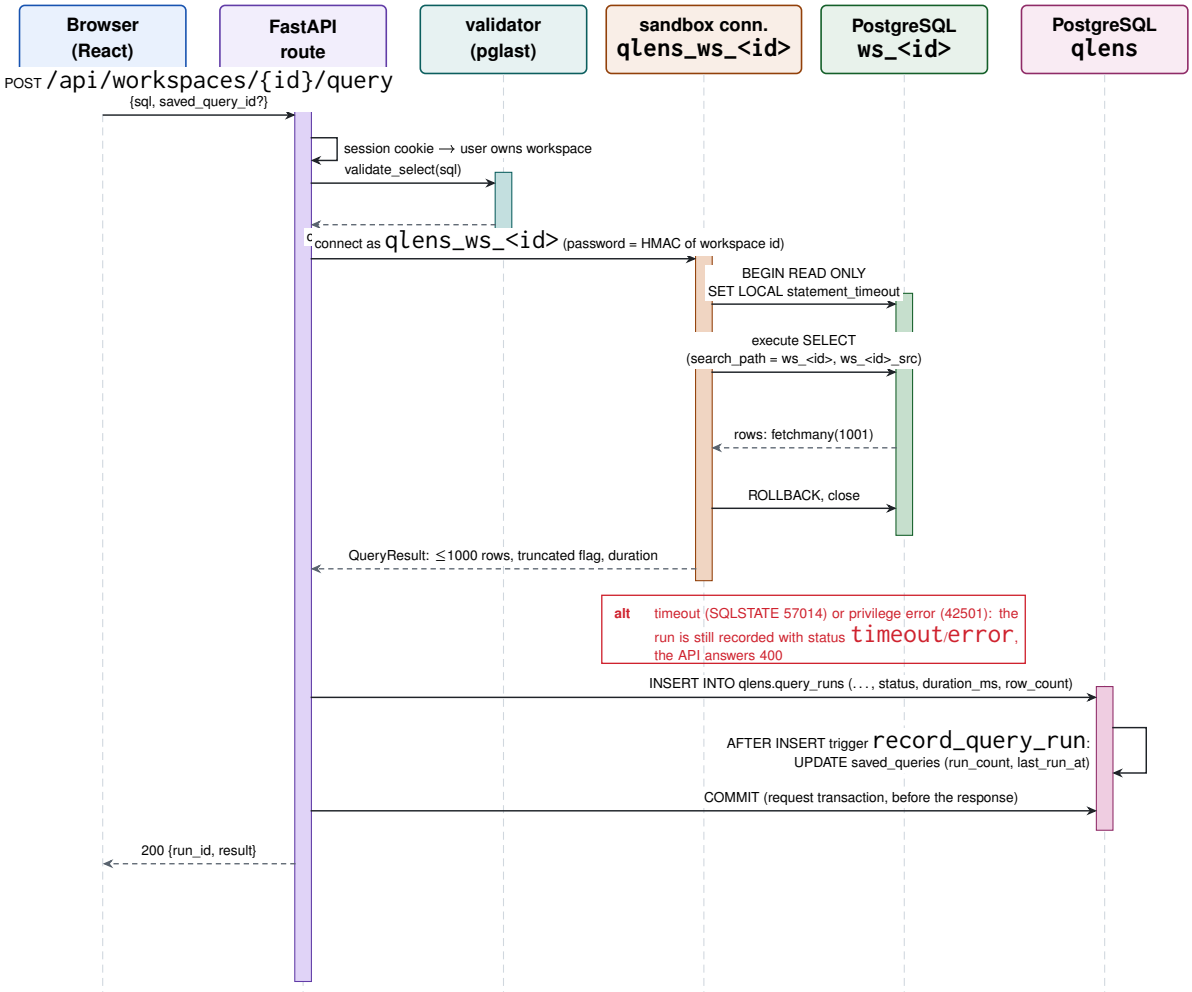


Figure 3.8: Sequence of a user query through the sandbox. The query never runs on the application's pooled connection: it runs on a separate login as the workspace role, whose privileges reach only that workspace's schemas, inside a read-only transaction with a statement timeout and a row cap. The run is then recorded on the pooled connection, where a trigger maintains the saved query's statistics. (Query runner specified; the role, its settings, `query_runs` and the trigger are built and tested.)

3.3.7 Security and Isolation

QLens accepts arbitrary SQL from the user, so its security model is built on privileges that the database itself enforces, in layers.

Per-workspace roles. Creating a workspace runs, inside the request transaction, a `CREATE ROLE` for a `LOGIN` role dedicated to that workspace, with `NOINHERIT`, `CONNECTION LIMIT 3`, and the following settings stored on the role: `statement_timeout = '5s'`, `default_transaction_read_only = on`, `idle_in_transaction_session_timeout = '10s'` and a `search_path` of the workspace's own schemas. Because the settings belong to the role, they apply to every session regardless of what the application code remembers to do. The role is then granted `USAGE` on the workspace's schemas and `SELECT` on their tables, with `ALTER DEFAULT PRIVILEGES` so tables created later are covered. It holds no privileges on any other workspace's schemas and none on the `qlens`

schema, so a query such as `SELECT * FROM qlens.users` fails with a permission error. Creation is atomic with the workspace row, and deletion drops the schemas, owned objects and role together.

Query path. Figure 3.8 shows what happens when a user runs a query. The session cookie is resolved to a user, the workspace is checked to belong to that user, the query runs on a sandbox connection as the workspace role, and the outcome (status, duration, row count or error) is written to `query_runs` by the application connection. The role's read-only setting means that even a statement such as `DROP TABLE` or `UPDATE` fails inside the database; no keyword filtering is involved, which avoids the usual bypasses of text filters. Identifiers that come from user data or from the proposal are always passed through `psycopg`'s identifier quoting and values are always bound parameters.

Passwords and sessions. Passwords are hashed with Argon2 (via `argon2-cffi`), a memory-hard algorithm designed to make offline guessing expensive. A login creates a random token, sends it to the browser in an `httpOnly` cookie and stores only its SHA-256 digest as the primary key of `sessions`; a `CHECK` enforces the 32-byte digest length. A copy of the database therefore cannot be replayed as live sessions, and logging out or deleting a user removes the sessions by cascade. Emails are stored in lower case behind a `CHECK (email = lower(email))`, which makes the `UNIQUE` constraint effectively case-insensitive.

3.3.8 Triggers and Views

Two kinds of database-side logic keep derived data correct regardless of which code path writes it.

Triggers. The function `touch_updated_at` stamps `updated_at` on every `UPDATE` of workspaces, datasets and saved queries, ignoring whatever value the client sent. It uses `clock_timestamp()` rather than `now()`, because `now()` is frozen at the start of the transaction and two updates inside one transaction would look simultaneous.

The function `record_query_run` (Listing 3.5) maintains the usage counters of a saved query. It is an `AFTER INSERT` row trigger on `query_runs` whose `WHEN` clause filters before the function is called: only successful runs that belong to a saved query count. Failed and timed-out runs are kept as history but do not increment `run_count`. The counters therefore cannot drift from the history table, and the application never has to remember to update them.

Listing 3.5: The usage-counter trigger (from `002_logic.sql`)

```
CREATE FUNCTION qlens.record_query_run() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
    UPDATE qlens.saved_queries
       SET run_count        = run_count + 1,
```

```
            last_run_at      = NEW.created_at,
            last_duration_ms = NEW.duration_ms
     WHERE id = NEW.saved_query_id;
    RETURN NULL;   -- AFTER trigger: return value is ignored
END;
$$;

CREATE TRIGGER query_runs_record AFTER INSERT ON qlens.query_runs
    FOR EACH ROW WHEN (NEW.saved_query_id IS NOT NULL AND NEW.status = 'ok')
    EXECUTE FUNCTION qlens.record_query_run();
```

Views. Three views are read directly by the API. `v_workspace_summary` produces one row per workspace for the dashboard: its stage (`empty` until a dataset exists, then the dataset status), the size of the dataset, the number of tables in the applied design and the latest activity. It uses two `LEFT JOIN LATERAL` aggregates; an aggregate without `GROUP BY` always returns exactly one row, so a workspace with no dataset or no queries is never dropped. `v_slow_queries` selects the successful runs of the last 30 days whose duration is at or above the workspace's 90th percentile, computed with `percentile_cont(0.9) WITHIN GROUP`.

`v_saved_query_trend` (Listing 3.6) uses window functions to give each saved query a timing history: the run number, the previous duration and a moving average over the current and four preceding runs. The row id is a tie-breaker in the window ordering, so runs with identical timestamps still get a stable order. The performance lab uses this view to show whether a query got faster after an index experiment.

Listing 3.6: Timing trend of saved queries with window functions (from `002_logic.sql`)

```
CREATE VIEW qlens.v_saved_query_trend AS
SELECT saved_query_id,
       id AS run_id,
       row_number() OVER (PARTITION BY saved_query_id ORDER BY created_at, id) AS
    run_number,
       created_at,
       duration_ms,
       lag(duration_ms) OVER (PARTITION BY saved_query_id ORDER BY created_at, id) AS
    prev_ms,
       avg(duration_ms) OVER (PARTITION BY saved_query_id ORDER BY created_at, id
                              ROWS BETWEEN 4 PRECEDING AND CURRENT ROW) AS moving_avg_ms
FROM qlens.query_runs
WHERE saved_query_id IS NOT NULL
  AND status = 'ok'
  AND duration_ms IS NOT NULL;
```

Summary of implementation status. Table 3.4 relates the design components above to what exists at the time of writing.

Table 3.4: Status of the designed components

Component	Notes	Status
Metadata schema, triggers, views	21 tables, generated columns, partial unique indexes	Built
Import and type inference	CSV and XLSX, `try_date`, column profile	Built
Dependency and key discovery	`check_fd`, `is_unique`, noise suppression	Built
Duplicate detection and fixes	`pg_trgm` with GIN, audit log, accept all	Built
FD algebra and 3NF synthesis	Closure, minimal cover, Bernstein, property tests	Built
Proposal and DDL generation	Surrogate keys, FK substitution, naming	In progress
Apply and lossless verification	Single transaction, `EXCEPT ALL`	In progress
Query sandbox and history	Role-based isolation, saved queries	In progress
Performance lab	`EXPLAIN ANALYZE`, index experiments	Planned
Step-through visualizer, assistant		Planned

CHAPTER 4

Conclusion

4.1 What Was Achieved

QLens set out to answer one practical question: can a person who only owns a messy spreadsheet be taken, step by step and with every step explained, to a verified third normal form database? The implemented system shows that the answer is yes for the class of data it targets, namely a single flat table of up to 60 columns and a few thousand rows.

The pipeline that exists today runs from a CSV or XLSX upload to a proposed 3NF design. Type inference, candidate-key search and functional-dependency discovery all execute inside PostgreSQL, through the PL/pgSQL functions `qlens.check_fd` and `qlens.is_unique`, so the data never leaves the database to be analysed. On the bundled coaching-centre fee register (3,000 rows, 8 columns) the complete analysis takes about 0.3 seconds. Near-duplicate detection uses `pg_trgm` with a GIN index on a temporary table and handles 5,000 distinct values in about one second [9]. The planted errors in that sample, eight rows with variant spellings of four student names and five rows carrying a teacher's superseded phone number, are found, shown to the user, and repaired with an audit trail in `dataset_fixes`.

The normalisation step rests on a small, separately tested algebra module: attribute closure, minimal cover and Bernstein's synthesis [4], with a lossless-join relation added whenever no synthesised relation contains a key. It was checked by property-based tests over thousands of random dependency sets rather than by a handful of hand-picked examples. For the fee register the result after cleanup is the expected four-table design: `students`, `batches`, `teachers` and a slim `fee_register` that carries only the student reference, month, amount and payment mode, with a composite unique key on student and month.

Around this pipeline sits a metadata schema of 21 tables that makes use of most of what a course on relational systems introduces: generated columns, `CHECK` constraints that encode business rules (for example, an applied proposal must have passed its lossless check), partial unique indexes that allow at most one draft and one applied proposal per dataset, triggers for timestamps and for recording query runs, and views built with window functions for slow-query and trend reporting. Each workspace is isolated by its own `LOGIN` role and its own pair of schemas, which is the mechanism later used to sandbox user queries. The backend carries 212 automated tests in its latest full run, the frontend has its own passing suite, and the interface has a pure-black dark theme and a light theme.

4.2 What Was Learnt

Several textbook ideas looked different once they had to run against real data.

Dependencies are statistical before they are logical. Theory defines a functional dependency as holding or not holding. A spreadsheet of 3,000 rows never behaves that way: the dependency {`student_phone`} → `student_name` fails in exactly eight rows because of typos. Recording `violating_rows` and a generated `violation_ratio` for every candidate, and letting the user decide whether a near-miss is a real rule with dirty data or a coincidence, turned out to be the central design decision of the analysis stage.

Normalisation needs clean data first. Bernstein's algorithm is only correct for the dependencies it is given. If the dependency from phone to name is rejected because of eight misspelt rows, the design silently keeps a redundant column. The order in the product (profile, find duplicates, repair, then analyse again) follows from this and is not a matter of taste.

Let the database do set-oriented work. Checking a dependency is a single `GROUP BY ... HAVING count(DISTINCT ...) > 1`. Running it inside the server avoided shipping rows to Python, and with a bound on the size of the left-hand side the number of candidate checks stays small enough to finish in a fraction of a second.

Constraints are documentation that executes. Encoding rules as `CHECK`, `UNIQUE` and partial indexes caught several of our own mistakes during development, long before any application code could have hidden them. The index comments in the DDL, each naming the query it serves, also made it simple to decide what *not* to index.

Lossless-join is a testable property. The theorem that a Bernstein decomposition is lossless becomes, in practice, a query: rebuild the join of the new tables and compare it with the staged data using `EXCEPT ALL` in both directions. Using the multiset form matters, because plain `EXCEPT` would hide duplicate rows that the decomposition might have lost or multiplied.

4.3 Limitations

We prefer to state the boundaries of the present work plainly.

- **Build status.** The apply step (creating the normalised tables in one transaction and running the lossless check), the design canvas and the query sandbox interface were still being finished when this report was written. The performance lab, the step-through visualiser and the English-language assistant are designed, and the metadata tables for them exist, but they are not yet implemented.
- **Bounded dependency search.** Left-hand sides are limited to at most two columns, and candidate keys to at most three. Real tables can have dependencies with three determining columns; these are not found. The bound keeps the search cheap in practice but is a deliberate loss of completeness.

- **Heuristics for noise.** Deciding which approximate dependencies deserve to be shown relies on thresholds on the violation ratio and on the number of repeated groups. A column with nearly unique values determines almost everything trivially, and such results are filtered out by rule, not by principle. On unusual data these rules may hide a real dependency or surface a spurious one, which is why the user always has the final say.
- **Single table, bounded size.** QLens normalises one flat table at a time. Multi-sheet workbooks, multivalued dependencies (4NF) and join dependencies (5NF) are outside its scope, and the target is thousands of rows rather than millions.
- **Fuzzy matching is not understanding.** Trigram similarity finds spellings that look alike; it cannot know that two distinct people share a name. Every merge is therefore a proposal with a confidence label and is recorded so that it can be audited.
- **Evidence of effectiveness.** The numbers in this report come from three synthetic datasets with planted problems and from automated tests. No study with real users has been carried out.

4.4 Future Work

Design canvas with a validated assistant. The next milestone is an interactive entity–relationship canvas where the user can rename tables, move a column, or split a table and see the effect on keys and foreign keys immediately. On top of it we plan an assistant powered by the Gemini API that accepts instructions such as “separate the teacher details”. The assistant would not write to the database. It would emit structured tool calls (rename, move column, add key), each of which is validated against the same constraint rules as manual edits, shown as a diff, and applied only after confirmation. Invalid or lossy requests are rejected by the validator and are never left to the model’s judgement.

Performance lab. The `query_runs` table already records duration and row counts, and `v_slow_queries` ranks the slowest statements. The lab will present `EXPLAIN (ANALYZE, BUFFERS)` output as a tree, compare plans before and after an index is added, and suggest indexes from the filters and joins that actually occur in the history.

Step-through visualiser. A teaching mode that evaluates a query one operator at a time (scan, filter, join, group, project) and shows the intermediate relation after each, so that a learner can connect the SQL text with relational algebra.

Larger dependency search and 4NF. Allowing a wider left-hand side with pruning by minimality, sampling for large tables, and detecting multivalued dependencies would extend the design stage towards BCNF and 4NF.

Deployment. QLens currently runs locally against a private PostgreSQL cluster. A hosted version would need per-tenant resource limits, connection pooling and scheduled clean-up of idle workspace schemas, all of which the per-workspace role design was chosen to make possible.

References

[1] E. F. Codd, "A relational model of data for large shared data banks," *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, 1970.

[2] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. McGraw-Hill, 2019.

[3] H. Garcia-Molina, J. D. Ullman, and J. Widom, *Database Systems: The Complete Book*, 2nd ed. Pearson Prentice Hall, 2009.

[4] P. A. Bernstein, "Synthesizing third normal form relations from functional dependencies," *ACM Transactions on Database Systems*, vol. 1, no. 4, pp. 277–298, 1976.

[5] The PostgreSQL Global Development Group, "PostgreSQL 18 documentation," https://www.postgresql.org/docs/18/, 2025.

[6] Meta Open Source, "React documentation," https://react.dev/, 2025.

[7] S. Ramírez, "FastAPI documentation," https://fastapi.tiangolo.com/, 2025.

[8] D. Varrazzo, "Psycopg 3: PostgreSQL database adapter for Python," https://www.psycopg.org/psycopg3/docs/, 2025.

[9] The PostgreSQL Global Development Group, "pg_trgm: Support for similarity of text using trigram matching," PostgreSQL 18 Documentation, Appendix F, https://www.postgresql.org/docs/18/pgtrgm.html, 2025.

CHAPTER A

Appendix

A.1 Complete DDL of Selected Tables

The statements below are copied from the first migration of the metadata schema (`001_schema.sql`). They cover the tables that carry the workspace, the imported dataset, the discovered dependencies, the design proposals and the query history.

Listing A.1: Workspaces, datasets and dataset columns.

```
CREATE TABLE qlens.workspaces (
    id         bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    owner_id   bigint NOT NULL REFERENCES qlens.users (id) ON DELETE CASCADE,
    name       text NOT NULL CHECK (length(name) BETWEEN 1 AND 60),
    created_at timestamptz NOT NULL DEFAULT now(),
    updated_at timestamptz NOT NULL DEFAULT now(),
    UNIQUE (owner_id, name)
);

CREATE TABLE qlens.datasets (
    id               bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    workspace_id     bigint NOT NULL UNIQUE
                     REFERENCES qlens.workspaces (id) ON DELETE CASCADE,
    original_filename text NOT NULL,
    source           text NOT NULL CHECK (source IN ('upload', 'sample')),
    file_size_bytes  integer NOT NULL CHECK (file_size_bytes >= 0),
    row_count        integer NOT NULL CHECK (row_count >= 0),
    column_count     smallint NOT NULL CHECK (column_count BETWEEN 1 AND 60),
    status           text NOT NULL DEFAULT 'imported'
                     CHECK (status IN ('imported', 'analyzed', 'applied')),
    created_at       timestamptz NOT NULL DEFAULT now(),
    updated_at       timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE qlens.dataset_columns (
    id             bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    dataset_id     bigint NOT NULL REFERENCES qlens.datasets (id) ON DELETE CASCADE,
    ordinal        smallint NOT NULL CHECK (ordinal >= 1),
    name           text NOT NULL CHECK (name ~ '^[a-z_][a-z0-9_]{0,62}$'),
    label          text NOT NULL,
    inferred_type  text NOT NULL
                   CHECK (inferred_type IN ('integer', 'numeric', 'boolean', 'date',
    'text')),
    null_count     integer NOT NULL DEFAULT 0 CHECK (null_count >= 0),
    distinct_count integer NOT NULL DEFAULT 0 CHECK (distinct_count >= 0),
    sample_values  text[] NOT NULL DEFAULT '{}',
    UNIQUE (dataset_id, name),
    UNIQUE (dataset_id, ordinal)
);
```

Listing A.2: Functional dependencies and their left-hand-side columns.

```
CREATE TABLE qlens.functional_dependencies (
    id              bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    dataset_id      bigint NOT NULL REFERENCES qlens.datasets (id) ON DELETE CASCADE,
    rhs_column_id   bigint NOT NULL
                    REFERENCES qlens.dataset_columns (id) ON DELETE CASCADE,
    lhs_size        smallint NOT NULL CHECK (lhs_size BETWEEN 1 AND 2),
    row_count       integer NOT NULL CHECK (row_count >= 0),
    lhs_values      integer NOT NULL CHECK (lhs_values >= 0),
    repeated_groups integer NOT NULL CHECK (repeated_groups >= 0),
    violating_rows  integer NOT NULL CHECK (violating_rows >= 0),
    is_exact        boolean GENERATED ALWAYS AS (violating_rows = 0) STORED,
    violation_ratio numeric GENERATED ALWAYS AS
                    (violating_rows::numeric / NULLIF(row_count, 0)) STORED,
    accepted        boolean NOT NULL DEFAULT true,
    created_at      timestamptz NOT NULL DEFAULT now(),
    CHECK (violating_rows <= row_count)
);
CREATE INDEX functional_dependencies_dataset_idx
    ON qlens.functional_dependencies (dataset_id);
CREATE INDEX functional_dependencies_rhs_idx
    ON qlens.functional_dependencies (rhs_column_id);

CREATE TABLE qlens.fd_lhs_columns (
    fd_id     bigint NOT NULL
              REFERENCES qlens.functional_dependencies (id) ON DELETE CASCADE,
    column_id bigint NOT NULL
              REFERENCES qlens.dataset_columns (id) ON DELETE CASCADE,
    PRIMARY KEY (fd_id, column_id)
);
CREATE INDEX fd_lhs_columns_column_idx ON qlens.fd_lhs_columns (column_id);
```

Listing A.3: Schema proposals with the lossless-join result.

```
CREATE TABLE qlens.schema_proposals (
    id                bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    dataset_id        bigint NOT NULL
                      REFERENCES qlens.datasets (id) ON DELETE CASCADE,
    status            text NOT NULL
                      CHECK (status IN ('draft', 'applied', 'superseded')),
    created_at        timestamptz NOT NULL DEFAULT now(),
    applied_at        timestamptz,
    lossless_ok       boolean,
    staging_rows      integer CHECK (staging_rows >= 0),
    reconstructed_rows integer CHECK (reconstructed_rows >= 0),
    mismatch_rows     integer CHECK (mismatch_rows >= 0),
    CHECK (status <> 'applied' OR lossless_ok IS TRUE)
);
CREATE UNIQUE INDEX schema_proposals_one_draft_idx
    ON qlens.schema_proposals (dataset_id) WHERE status = 'draft';
CREATE UNIQUE INDEX schema_proposals_one_applied_idx
    ON qlens.schema_proposals (dataset_id) WHERE status = 'applied';
```

Listing A.4: Query history.

```
CREATE TABLE qlens.query_runs (
```

```
    id              bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    workspace_id    bigint NOT NULL
                    REFERENCES qlens.workspaces (id) ON DELETE CASCADE,
    saved_query_id bigint REFERENCES qlens.saved_queries (id) ON DELETE SET NULL,
    sql_text        text NOT NULL,
    source          text NOT NULL
                    CHECK (source IN ('typed', 'english', 'stepthrough', 'perf')),
    target          text NOT NULL CHECK (target IN ('main', 'lab')),
    status          text NOT NULL CHECK (status IN ('ok', 'error', 'timeout')),
    duration_ms     numeric(12, 3) CHECK (duration_ms >= 0),
    row_count       integer CHECK (row_count >= 0),
    error_message   text,
    created_at      timestamptz NOT NULL DEFAULT now(),
    CHECK ((status = 'ok') = (error_message IS NULL))
);
CREATE INDEX query_runs_workspace_time_idx
    ON qlens.query_runs (workspace_id, created_at DESC);
CREATE INDEX query_runs_saved_query_time_idx
    ON qlens.query_runs (saved_query_id, created_at)
    WHERE saved_query_id IS NOT NULL;
```

A.2 Sample Datasets

Three deterministic datasets ship with QLens so that every demonstration and every test sees identical numbers. All randomness comes from a single generator seeded with 2026, and the key counts are asserted again when the data is produced.

A.2.1 Coaching fee register

3,000 rows: 300 students for 10 months (April to January), one row per student per month.

- **Columns:** `Student Name`, `Student Phone`, `Batch`, `Teacher`, `Teacher Phone`, `Month`, `Fee Paid`, `Payment Mode`.
- **Intended dependencies:** {`student_phone`} → `student_name` and the reverse, {`student_name`} → `batch`, {`batch`} → `teacher`, {`teacher`} → `teacher_phone`; the key is (student, month). Eight batches share six teachers.
- **Planted anomalies:** four students (Riya Sharma, Aman Verma, Neha Singh, Karan Mehta) each appear in two rows under a lower-case spelling and a double-space spelling, giving exactly eight violating rows for the phone-to-name dependency. A genuine, different student named Priya Sharma has her own phone and must not be merged. One teacher, Mr. Gupta, has an old phone number in exactly five rows.
- **Other properties:** fee depends on the batch family (JEE 4000, NEET 3500, Foundation 2500); about 8% of rows are unpaid with fee 0 and an empty payment mode; about 15% of paid rows are partial payments. Phones are written as `+91 98765 43210` so that they are inferred as text.

A.2.2 Kirana store sales

600 bills with one to six distinct items each.

- **Columns:** `Bill No`, `Date` (written dd/mm/yyyy), `Customer`, `Customer Phone`, `Item`, `Category`, `Unit Price`, `Qty`, `Amount`.
- **Shape:** 80 customers, 40 items in 6 categories; `Amount` equals unit price times quantity in every row.
- **Planted anomalies:** `Toor Dal 1kg` carries an old price in exactly three rows (with a consistent amount), and the customer `Mohan Lal` is written `Mohan lal` in exactly three rows.

A.2.3 Hostel mess attendance

2,520 rows: 60 students for 14 days and 3 meals.

- **Columns:** `Room`, `Student`, `Hostel Block`, `Warden`, `Date` (yyyy-mm-dd), `Meal`, `Menu Item`, `Attended` (Yes or No).
- **Intended dependencies:** room determines block, block determines warden, and there is one menu item per (date, meal).
- **Planted anomaly:** the warden of Block B, `Mrs. Rao`, is written `Mrs Rao` in exactly two rows.

A.3 Screenshots

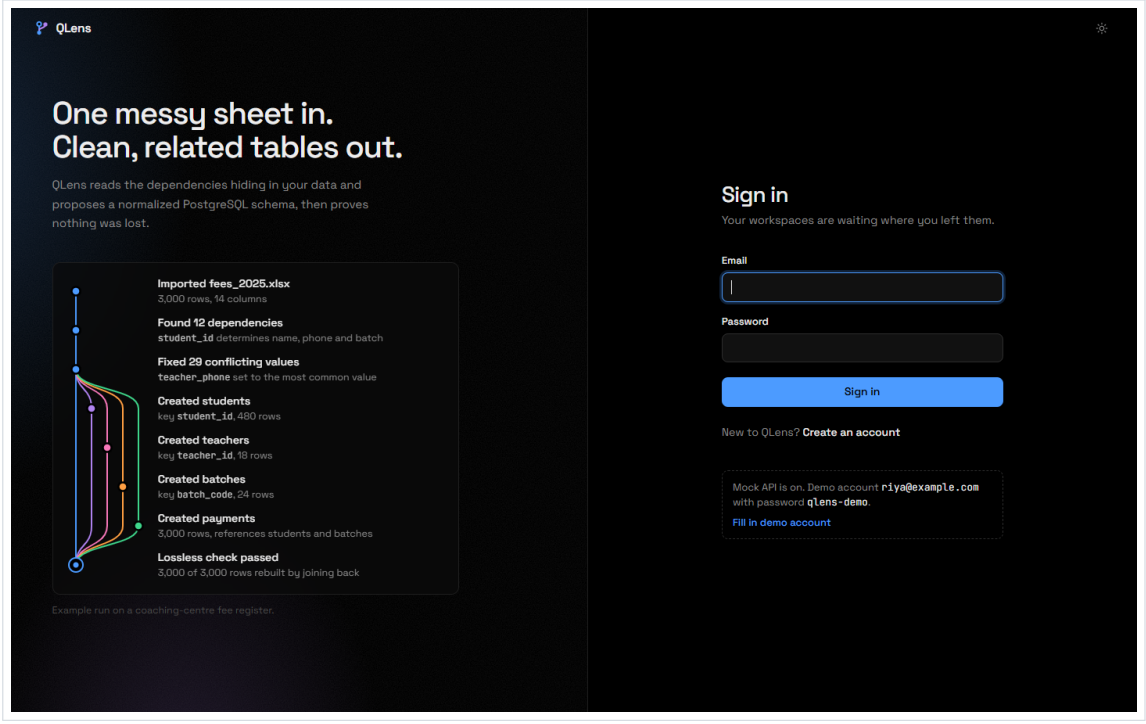


Figure A.1: Sign-in screen

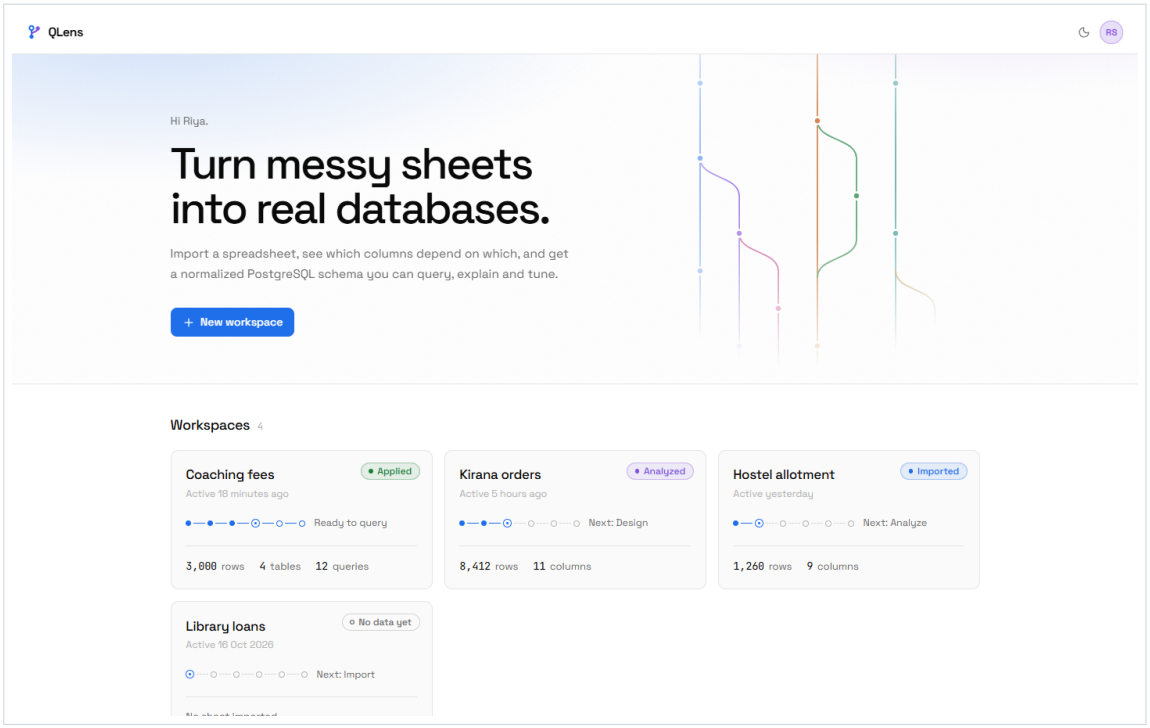


Figure A.2: Dashboard, light theme

A.4 Test Inventory

The backend tests live in `backend/tests` and run against a real PostgreSQL instance, not a mock. Table A.1 lists the number of test functions per file. Some functions are parametrised, so the number of cases collected by the runner (212 in the latest full run) is higher than the 172 functions counted here.

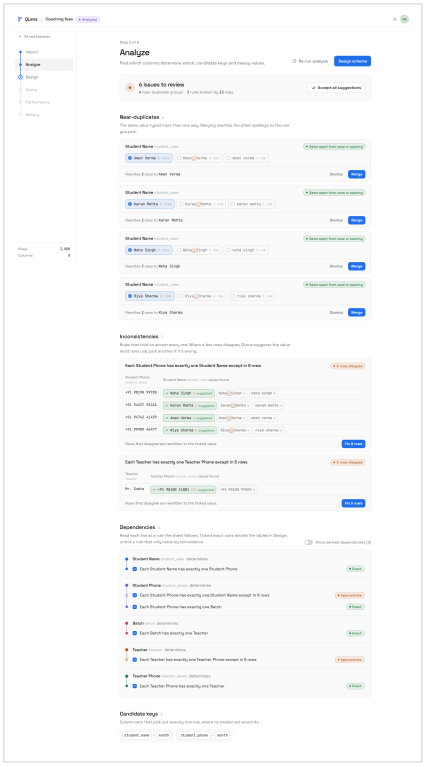


Figure A.3: Analyze, light theme

Table A.1: Backend test files and the number of test functions in each.

File	Tests	Covers
`test_cleanup.py`	35	near-duplicate groups, fixes, audit log, accept-all
`test_proposal.py`	21	3NF proposal, foreign-key substitution, generated DDL
`test_fd_algebra.py`	19	closure, minimal cover, Bernstein synthesis, property tests
`test_discover.py`	18	dependency and candidate-key discovery in SQL
`test_schema.py`	15	constraints, generated columns, triggers, views
`test_workspaces.py`	11	workspace roles and schemas
`test_stage.py`	10	staging tables and type inference
`test_parse.py`	9	CSV and XLSX parsing
`test_auth.py`	8	password hashing, sessions, cookies
`test_ingest_routes.py`	8	upload and import endpoints
`test_naming.py`	7	identifier and table naming
`test_samples.py`	6	sample dataset guarantees
`test_migrate.py`	3	migration runner
`test_conn_commit.py`	2	connection and commit behaviour
Total	**172**	